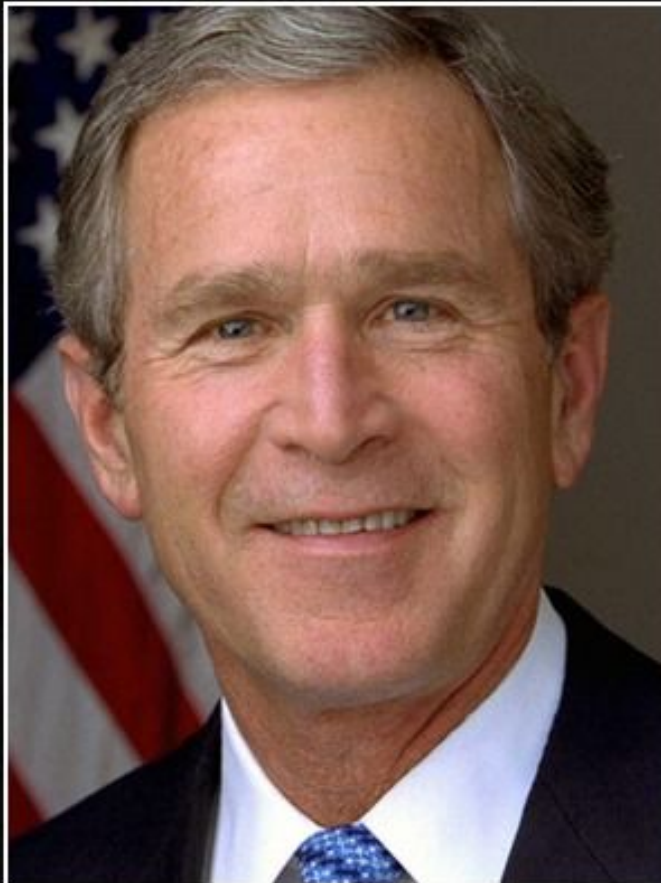


Moments Of Clarity in Machine Learning for Jet Physics

Rikab Gambhir

Based on [RG, Nachman, Thaler, [2205.03413](#)]
[RG, Nachman, Thaler, [2205.05084](#)]
[Ba, Dogra, RG, Tasissa, Thaler, [2302.12266](#)]
[RG, Thaler, Wu, WIP]
[RG, Larkoski, Thaler, [2410.05379](#)]
[RG, Osathapan, Thaler, [2403.08854](#)]



Rarely is the question asked: Is our
~~children~~ learning?
machines

— George W. Bush —

AZ QUOTES



Rarely is the question asked: Is our ~~children~~ learning?
machines

Yes. They've *already* learned!

Do you know physics?

Yes, I have a strong background in physics, particularly in areas like particle physics, collider physics and quantum field theory. I also have experience with advanced topics such as jet physics, machine learning applications in physics, and tools like Herwig, Pythia, and RAMBO. How can I assist you with a specific physics question or problem?

[Kasieczka et. al.; [1902.09914](#)]

	AUC	Acc	$1/\epsilon_B (\epsilon_S = 0.3)$			#Param
			single	mean	median	
CNN [16]	0.981	0.930	914±14	995±15	975±18	610k
ResNeXt [31]	0.984	0.936	1122±47	1270±28	1286±31	1.46M
TopoDNN [18]	0.972	0.916	295±5	382±5	378±8	59k
Multi-body N -subjettiness 6 [24]	0.979	0.922	792±18	798±12	808±13	57k
Multi-body N -subjettiness 8 [24]	0.981	0.929	867±15	918±20	926±18	58k
TreeNIN [43]	0.982	0.933	1025±11	1202±23	1188±24	34k
P-CNN	0.980	0.930	732±24	845±13	834±14	348k
ParticleNet [47]	0.985	0.938	1298±46	1412±45	1393±41	498k
LBN [19]	0.981	0.931	836±17	859±67	966±20	705k
LoLa [22]	0.980	0.929	722±17	768±11	765±11	127k
LDA [54]	0.955	0.892	151±0.4	151.5±0.5	151.7±0.4	184k
Energy Flow Polynomials [21]	0.980	0.932	384	633±31	729±13	1k
Energy Flow Network [23]	0.979	0.927	891±18	1063±21	1052±29	82k
Particle Flow Network [23]	0.982	0.932	891±18	1063±21	1052±29	82k
GoaT	0.985	0.939	1368±140	1549±208	1549±208	35k

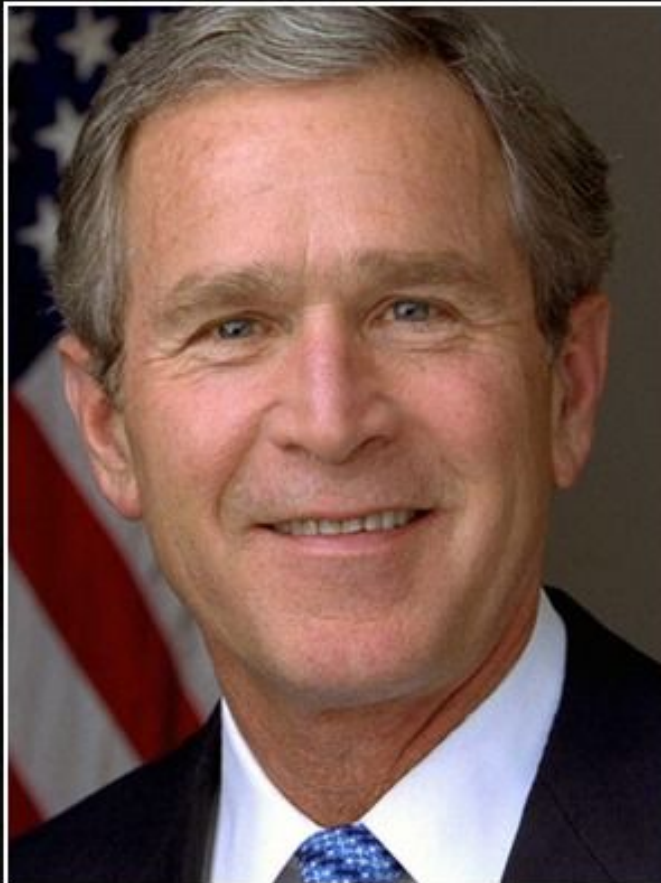
THE NOBEL PRIZE
IN PHYSICS 2024



John J. Hopfield Geoffrey E. Hinton

"for foundational discoveries and inventions
that enable machine learning
with artificial neural networks"

THE ROYAL SWEDISH ACADEMY OF SCIENCES



Rarely is the question asked: Is our
~~children learning?~~
~~machines~~
physicists using machines

— George W. Bush —

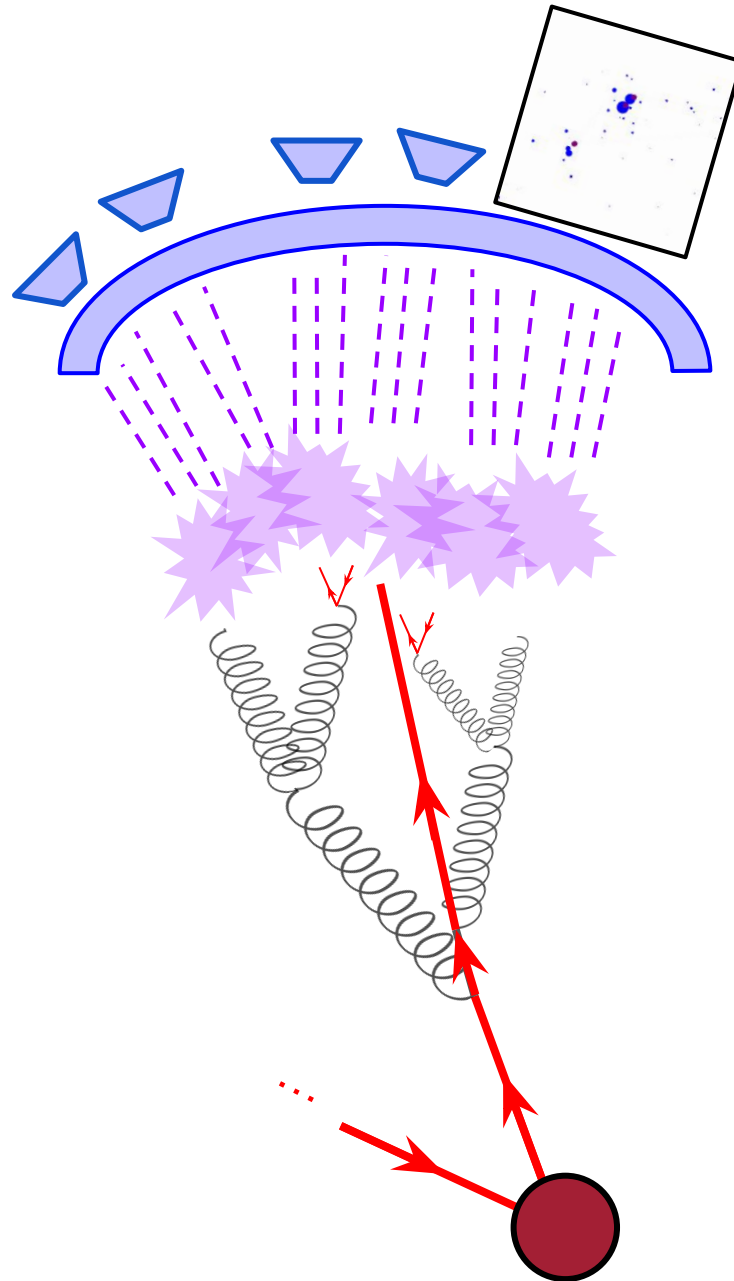
AZ QUOTES

Introduction

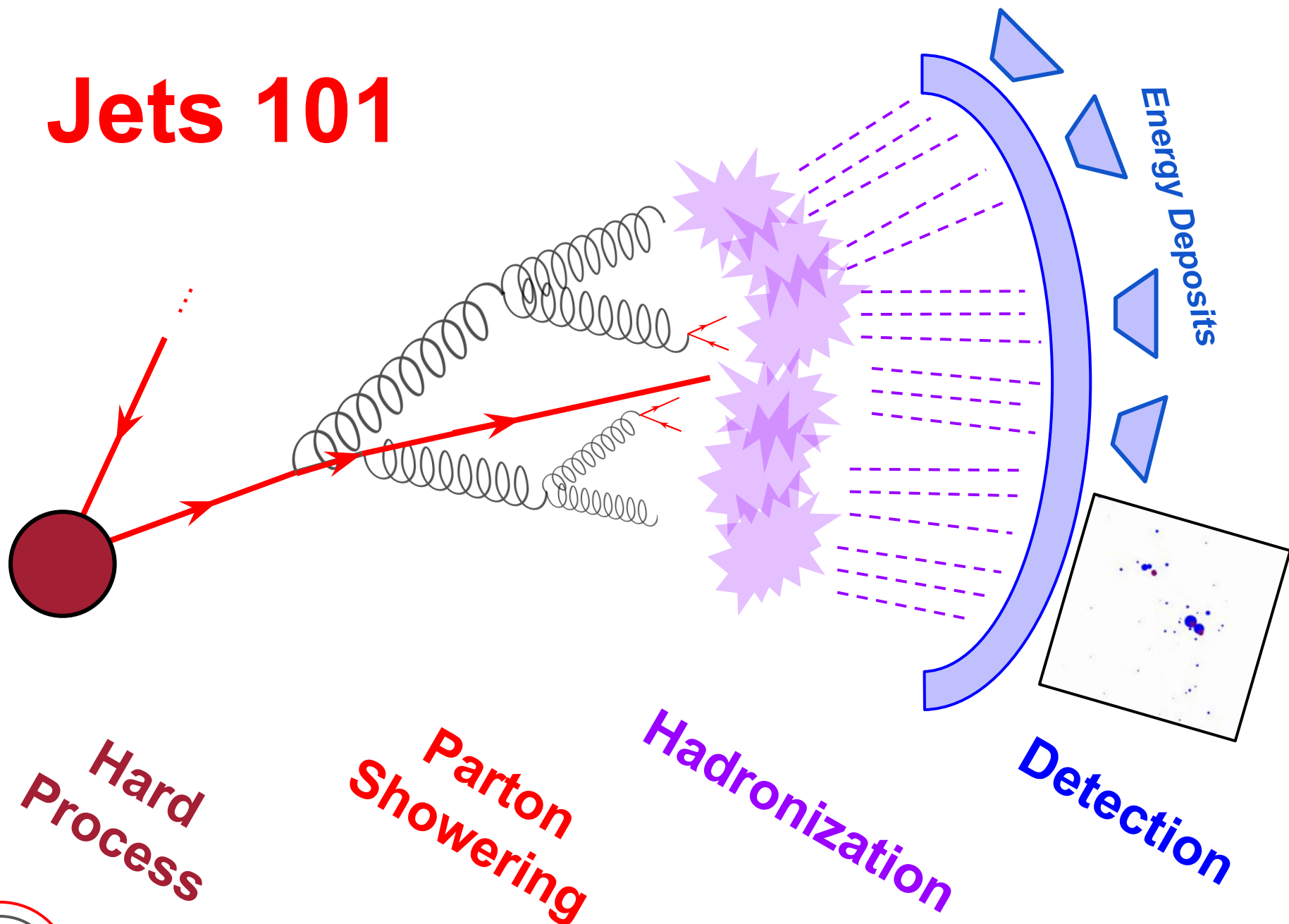
I want to study **jets** at the **Large Hadron Collider** (LHC). Jets are complicated!

I want to use **machine learning** (ML) to make my job easier, but with safeguards in place to make sure the physics I learn makes some sense!

This talk: 3 vignettes from my work in jet physics designing ML algorithms to give me exactly what I want.

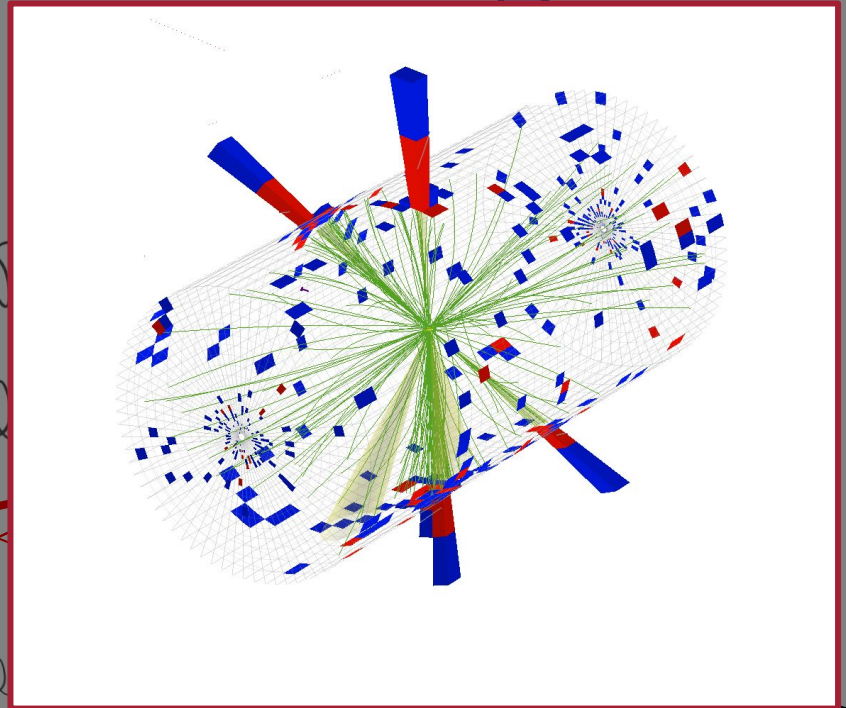


Jets 101

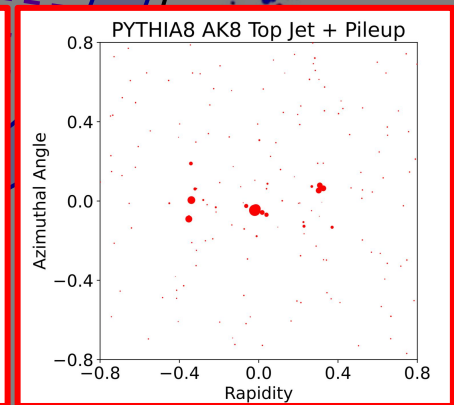
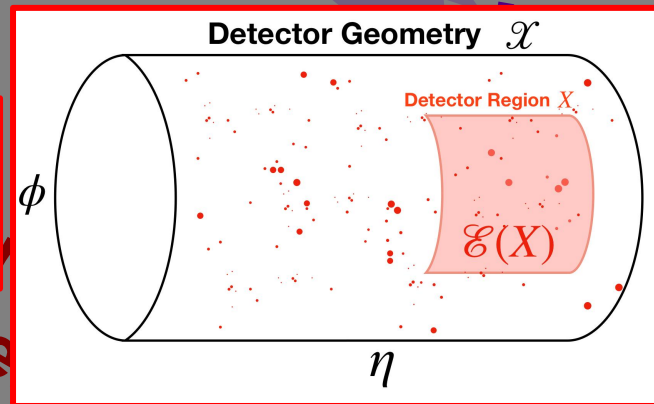


Jets 101

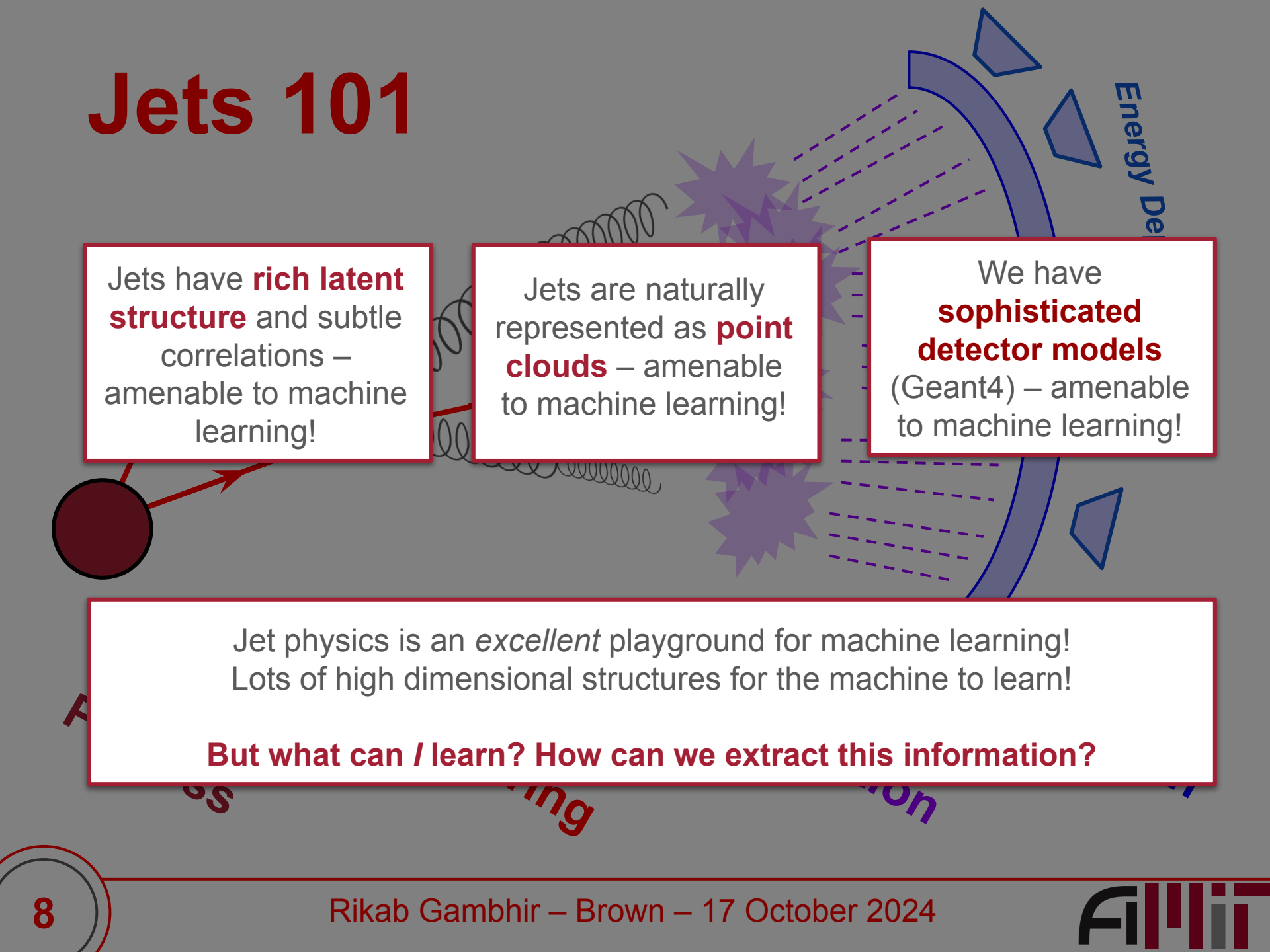
What a jet actually looks like
at the CMS detector



My theorist's picture of a jet



Jets 101



Jets have **rich latent structure** and subtle correlations – amenable to machine learning!

Jets are naturally represented as **point clouds** – amenable to machine learning!

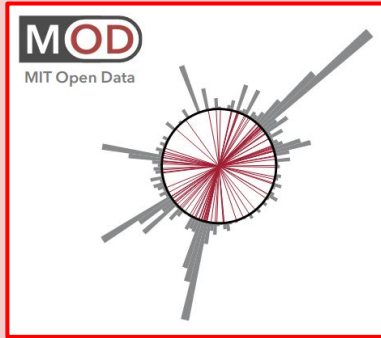
We have **sophisticated detector models** (Geant4) – amenable to machine learning!

Jet physics is an *excellent* playground for machine learning!
Lots of high dimensional structures for the machine to learn!

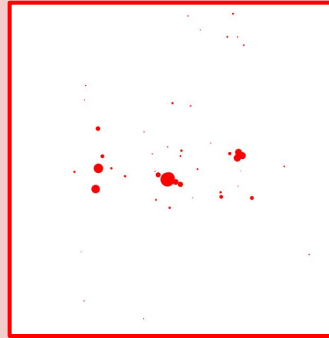
But what can I learn? How can we extract this information?

Things a **machine** can understand

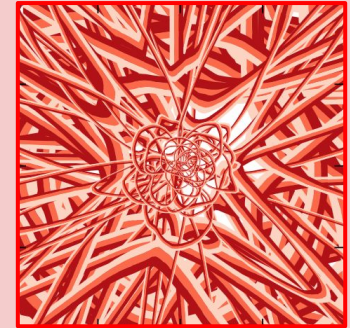
Sophisticated Detector Models



Complicated point clouds

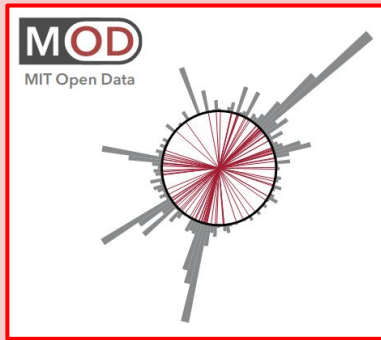


Huge Latent Spaces

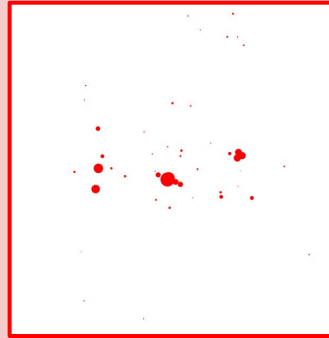


Things a **machine** can understand

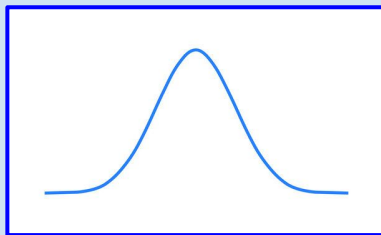
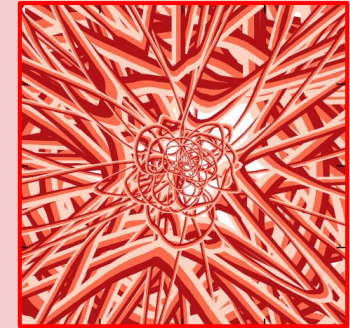
Sophisticated Detector Models



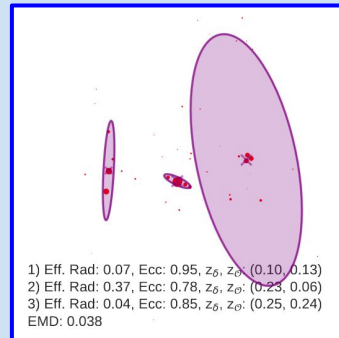
Complicated point clouds



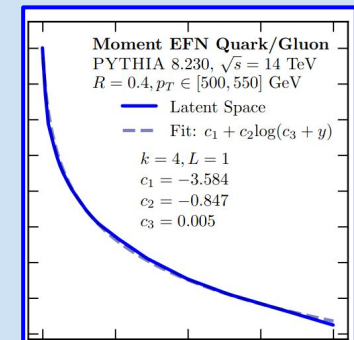
Huge Latent Spaces



Gaussians



Basic Kindergarten Shapes

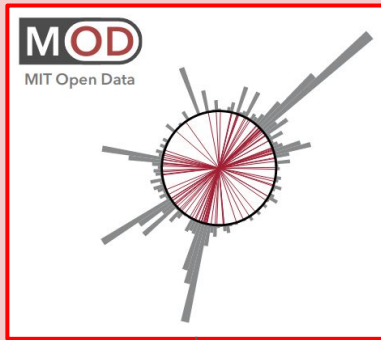


Addition, multiplication, and a few elementary functions

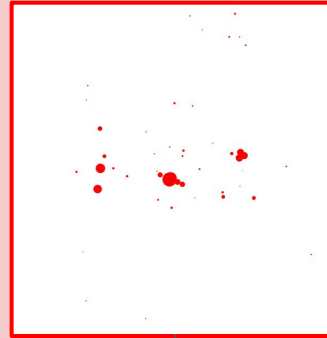
Things my **mortal physicist brain** can understand

Things a **machine** can understand

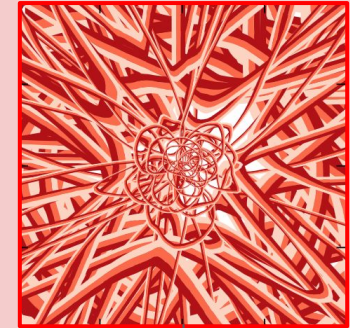
Sophisticated Detector Models



Complicated point clouds

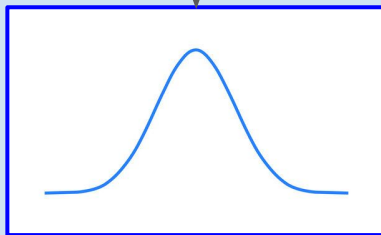


Huge Latent Spaces



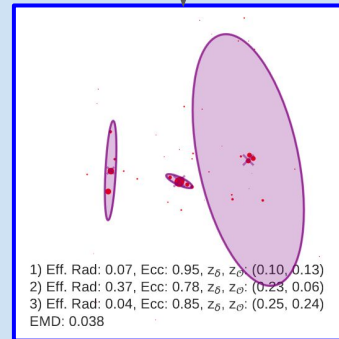
Interface: *Targeted and goal-motivated* network architectures and loss functions!

Using the
Gaussian
Ansatz and
DV Loss



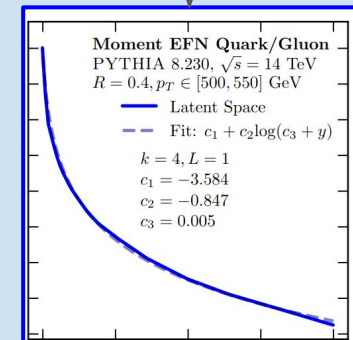
Gaussians

Using faithful
optimal transport



Basic Kindergarten Shapes

Using Moment
Pooling



Addition, multiplication, and a
few elementary functions

Things my **mortal physicist brain** can understand

Things a **machine** can understand

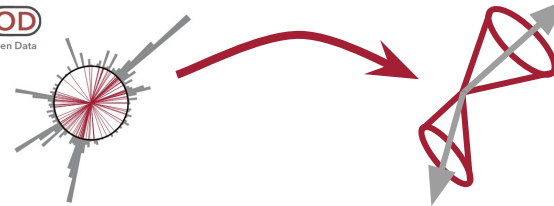
Sophisticated Detector Models

Complicated point clouds

Huge Latent Spaces



Learning **Uncertainties** the **Frequentist** Way



[RG, Nachman, Thaler, [2205.05084](#)]

[RG, Nachman, Thaler, [2205.05084](#)]

Can you Hear the **Shape** of a Jet?

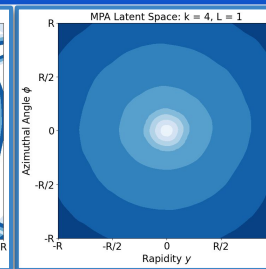
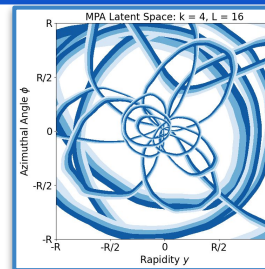


[Ba, Dogra, RG, Tasissa, Thaler, [2302.12266](#)]

[RG, Thaler, Wu, WIP]

[RG, Larkoski, Thaler, [2410.05379](#)]

The **Moments** of Clarity:
Streamlining Latent Spaces



[RG, Osathapan, Thaler, [2403.08854](#)]

Things my **mortal physicist brain** can understand

Part I

Learning **Uncertainties** the **Frequentist** Way: Calibration and Correlation

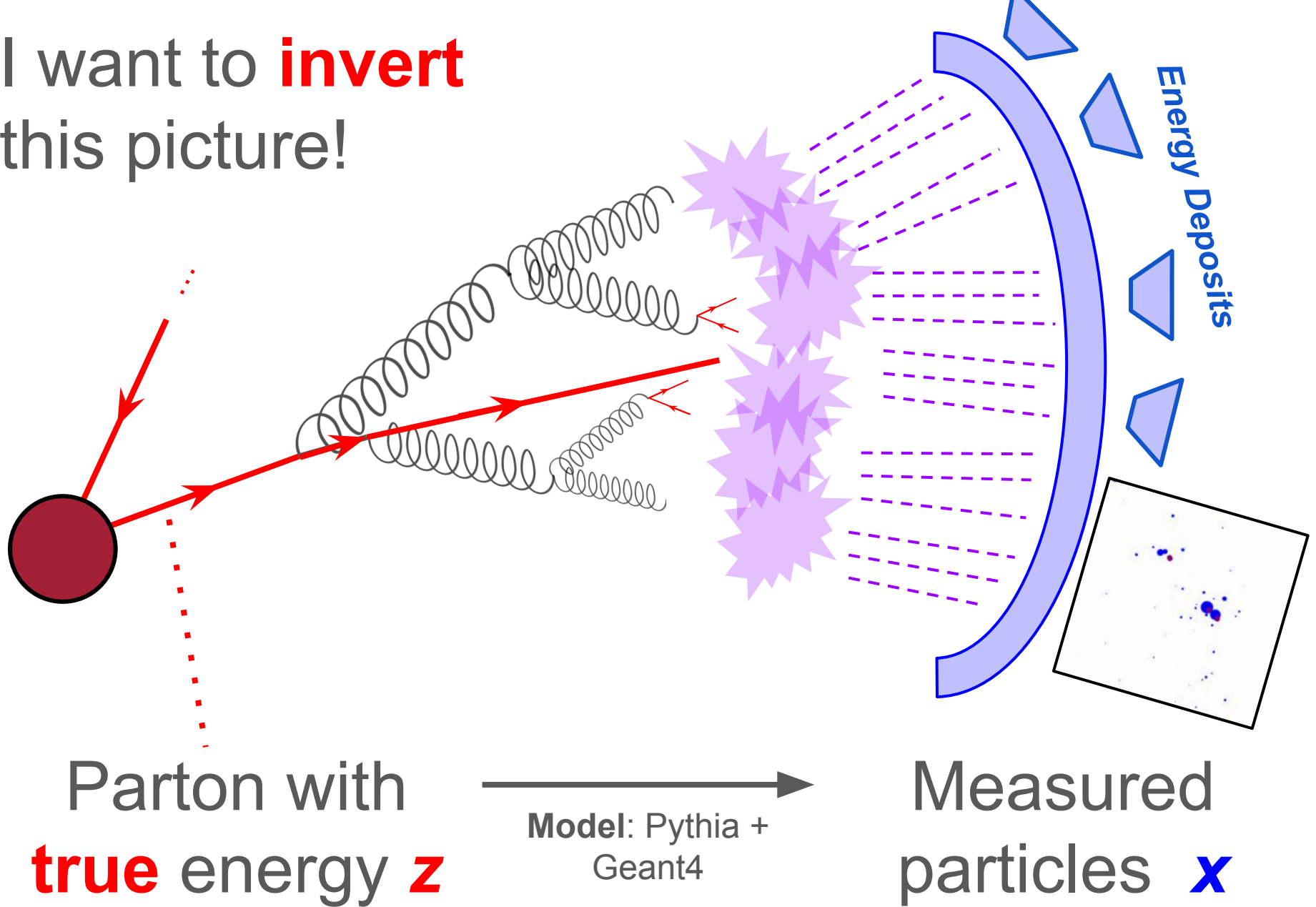


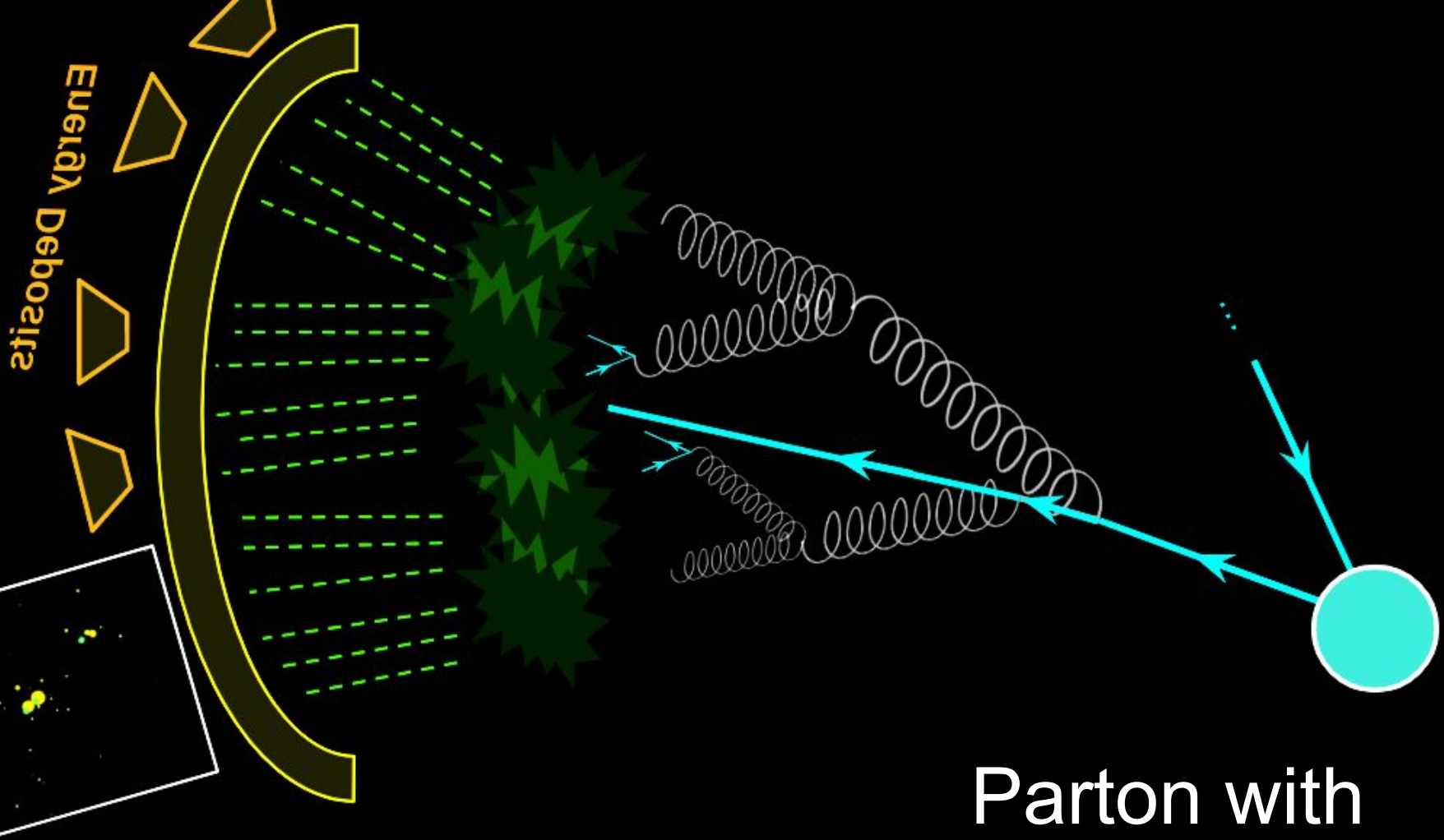
Download
our repo!

Try *pip install*
GaussianAnsatz



I want to **invert**
this picture!





Measured
particles x

→
What function does this?
Machine learn it!

Parton with
estimated
energy $\hat{z}$

Energy Deposits

Rich existing literature!

Simulation based inference & Uncertainty Estimation:

[Cranmer, Brehmer, Louppe 1911.01429;

Alaa, van der Schaar 2006.13707;

Abdar et. al, 2011.06225;

Tagasovska, Lopez-Paz, 1811.00908;

And many more!

Bayesian techniques:

[Jospit et. al, 2007.06823;

Wang, Yeung 1604.01662;

Izmailov et. al, 1907.07504;

Mitos, Mac Namee, 1912.1530;

And many more!

Measured
particles x

What function does this?
Machine learn it!

particle with
estimated
energy $\hat{z}$

Problem Statement

Given a training set of (x, z) pairs, can we machine learn a function f such that $f(x) = z$? With uncertainties?

Yes. Extremely easily. This is just bread-and-butter least-squares regression with a neural network g :

$$\operatorname{argmin}_g \mathbb{E}_{\text{train}}[(g(X) - Z)^2]$$

It's the first thing you learn how to do with ML!

Basic regression: Predict fuel efficiency 



Run in Google Colab



View source on GitHub



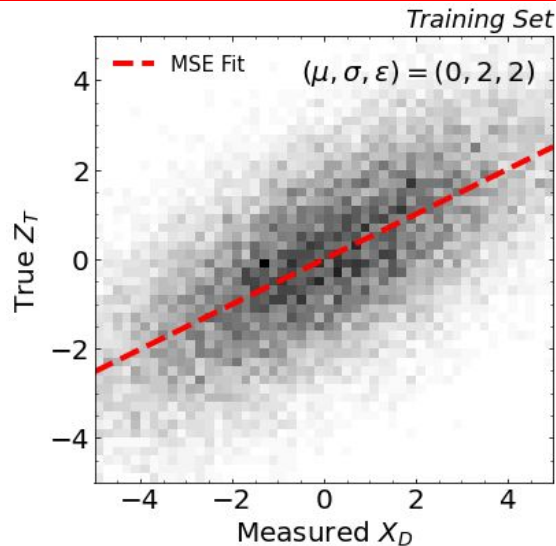
Download notebook

In a *regression* problem, the aim is to predict the output of a continuous value, like a price or a probability. Contrast this with a *classification* problem, where the aim is to select a class from a list of classes (for example, where a picture

Problem Statement

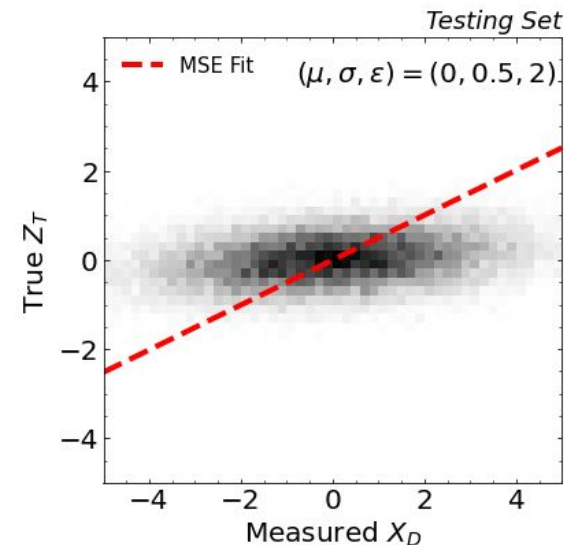
Not so fast!

Given a training set of (x, z) pairs, we learn a function f such that $f(x) = z$? With uncertainty.



Same
“detector” sim
 $p(x|z)$, only
different priors
 $p(z)$!

We can't apply
our calibration
universally.



It's the first thing

Upon closer inspection, MSE gives us prior dependent neural regressions!
Is this what we really want? Back to the drawing board!

In a regression problem, the aim is to predict the output of a continuous value, like a price or a probability. Contrast this with a classification problem, where the aim is to select a class from a list of classes (for example, where a picture

Calibration

Let's agree on what makes a good calibration, then design a loss for it!

1. **Closure**: On average, $f(x)$ should be correct for each x ! That is, f is **unbiased**.

$$b(z) = \mathbb{E}_{\text{test}}[f(X) - z | Z = z] = 0$$

2. **Universality**: $f(x)$ should not depend on the choice of sampling for z . That is, f is **prior-independent**.

f depends only on $p(x|z)$, and not $p(z)$

Likelihood: Detector simulation, noise model, etc

What if the detector simulation is imperfect? Ask me later!

Learning T

The **Donsker-Varadhan Representation (DVR)** of the KL divergence has been used in the statistics literature for mutual information estimation

$$\mathcal{L}_{\text{DVR}}[T] = - \left(\mathbb{E}_{P_{XZ}} [T] - \log \left(\mathbb{E}_{P_X \otimes P_Z} [e^T] \right) \right)$$

Interestingly, a nonlocal loss!

Strict bound on $I(X;Z)$

Minimized when

$$T(x, z) = \log \frac{p(x|z)}{p(x)} + c$$

What we want!

Unimportant

Lots of other losses also work, but DVR has very nice convergence properties - ask me later!

Inference using T

We can use a neural net to parameterize $T(x, z)$, and use standard gradient descent techniques to minimize the DVR loss. Then we can do ...

$$\hat{z}(x) = \operatorname{argmax}_z T(x, z)$$

Inference

$$[\hat{\sigma}_z^2(x)]_{ij} = - \left[\frac{\partial^2 T(x, z)}{\partial z_i \partial z_j} \right]^{-1} \Big|_{z=\hat{z}}$$

Gaussian Uncertainty Estimation

BUT!

- Maxima are hard to estimate – even *more* gradient descent!
- Second derivatives are sensitive to the choice of activations in T – ReLU spoils everything!

Inference using T

We can use a neural net to parameterize $T(x,z)$, and use standard gradient descent techniques to minimize the DVB loss. Then we can do

The **problems**:

- Maxima are hard to estimate – even *more* gradient descent!
- Second derivatives are sensitive to the choice of activations in T – ReLU spoils everything!

The **moment of clarity**: if something is hard to estimate, build it in!

- Maxima: Have an explicit term in the model to learn the maximum!
- Second derivatives: Have an explicit second order term to model Gaussian errors!

We solve both problems with the **Gaussian Ansatz**

The Gaussian Ansatz

Parameterize $T(x,y)$ in the following way (the **Gaussian Ansatz**):

$$\begin{aligned} T(x, z) = & A(x) \\ & + (z - B(x)) \cdot D(x) \\ & + \frac{1}{2} (z - B(x))^T \cdot C(x, z) \cdot (z - B(x)) \end{aligned}$$

Where $A(x)$, $B(x)$, $C(x,z)$, and $D(x)$ are learned functions. Then, if $D \rightarrow 0$, our inference and (Gaussian) uncertainties are given by ...

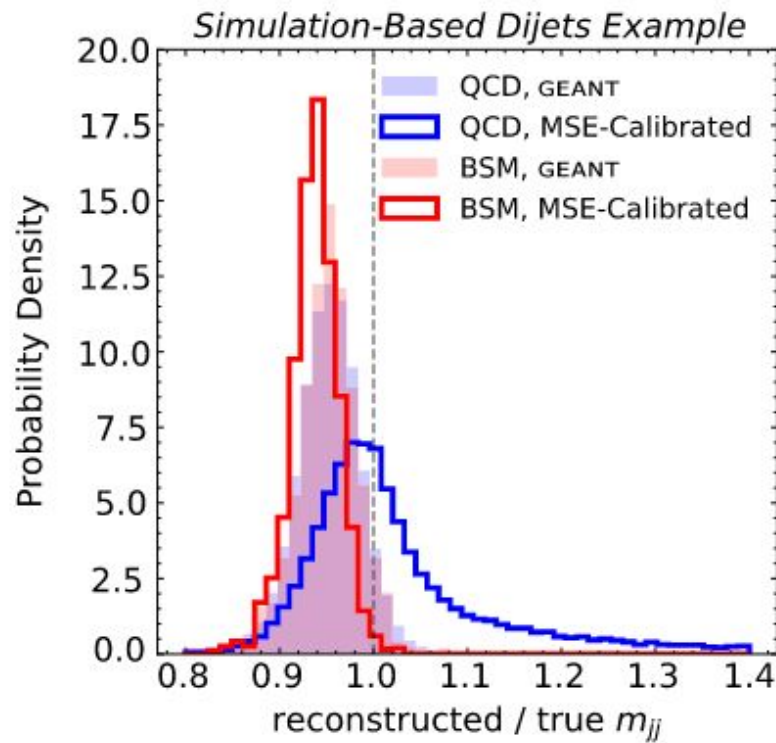
$$\hat{z}(x) = B(x) \qquad \hat{\sigma}_z^2(x) = -[C(x, B(x))]^{-1}$$

Our Maximum!

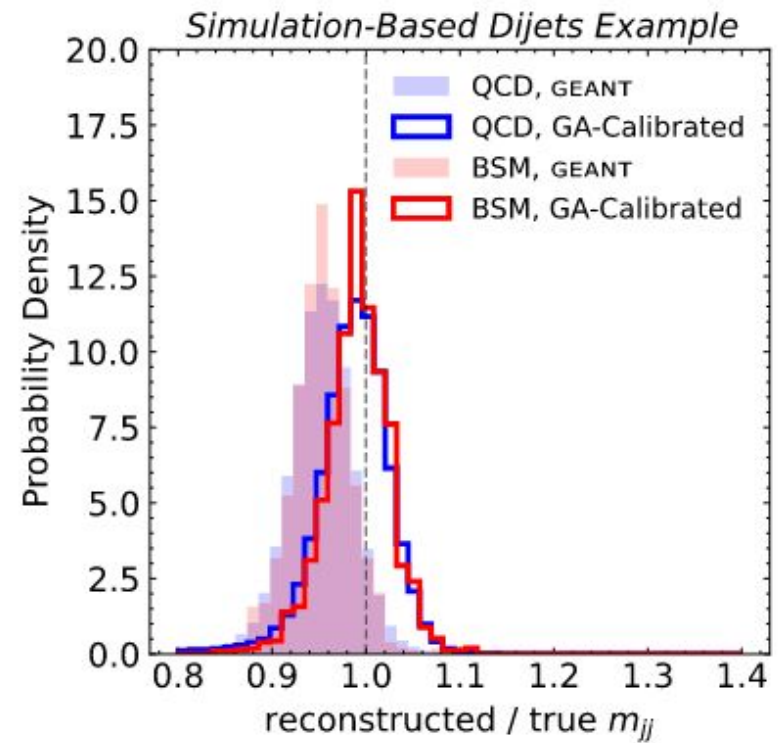
Our Second Derivative!

No additional post processing or numerical estimates required!

Example 1: QCD and BSM Dijets



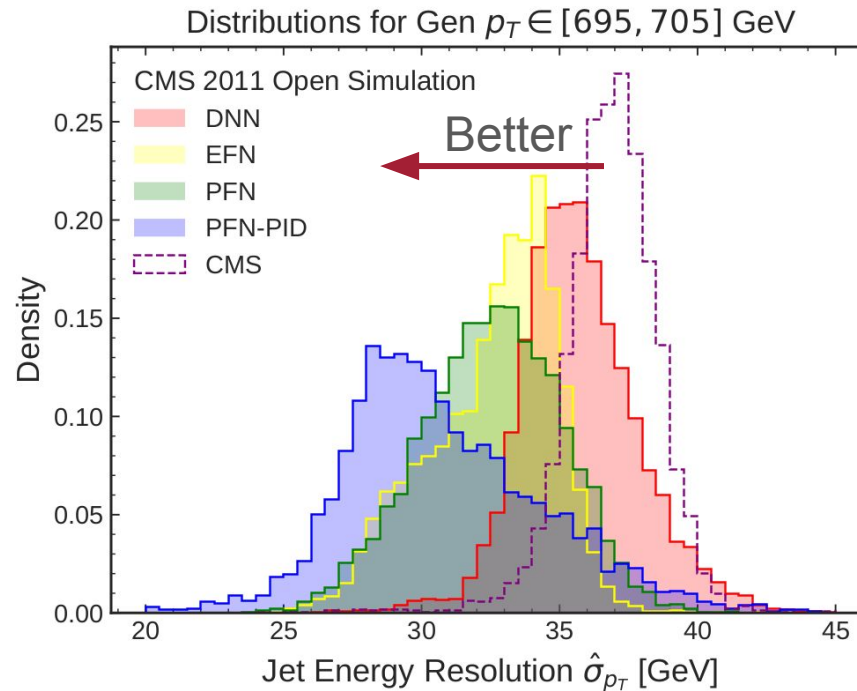
(Left) MSE-fitted network.



(Right) Gaussian Ansatz-fitted network

Clever loss function choice: *Prior dependence is built-in!*

Example 2: Jet Energy Resolution



Best of both worlds: Using ML to extract more information* than hand-crafted features, while still being able to extract resolutions in a prior-independent and unbiased way!

*I mean “information” literally. Ask me later about the cool information theory of the DV loss!

Part II

Can you Hear the **Shape** of a Jet?



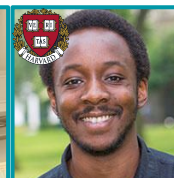
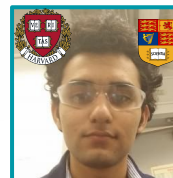
Download
SHAPER!

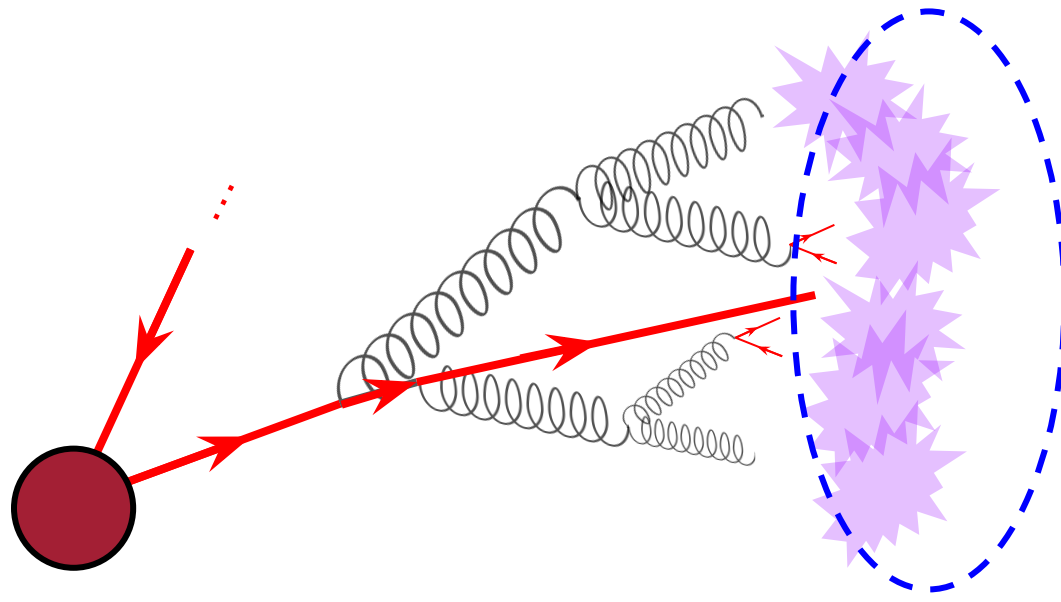


Download
SPECTER!

Try `pip install pyshaper`

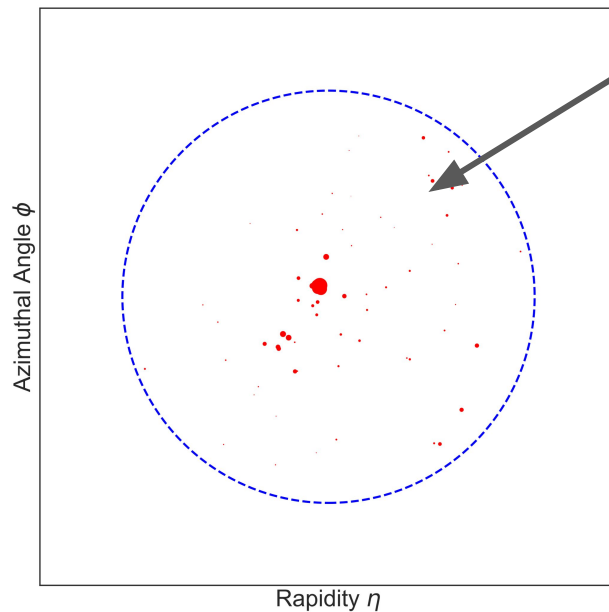
(New!) `pip install specterpy`



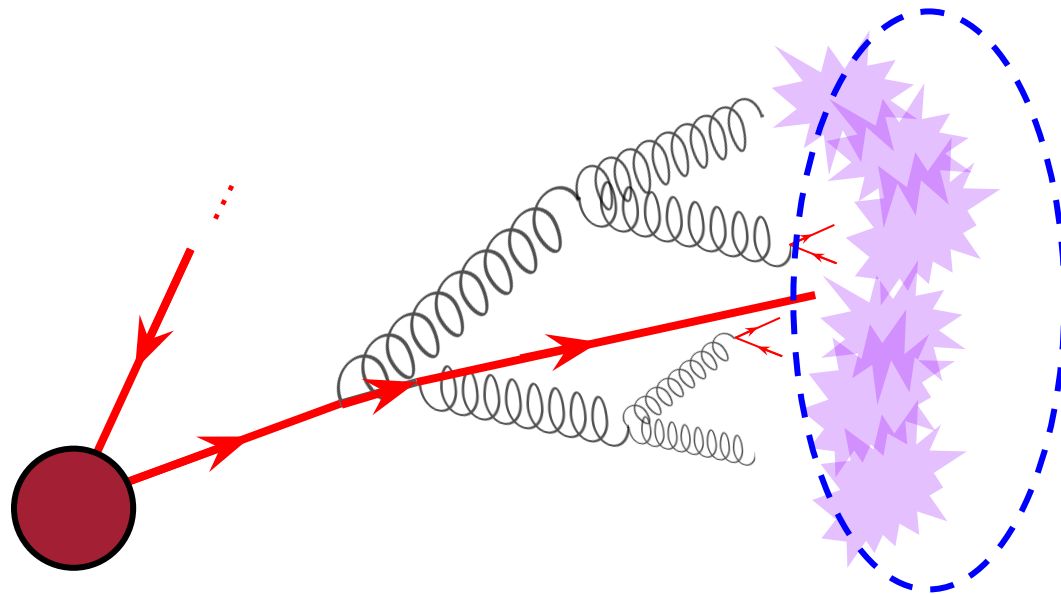


How “*big*” is this jet?

... What do we even mean by the “size” of a jet?

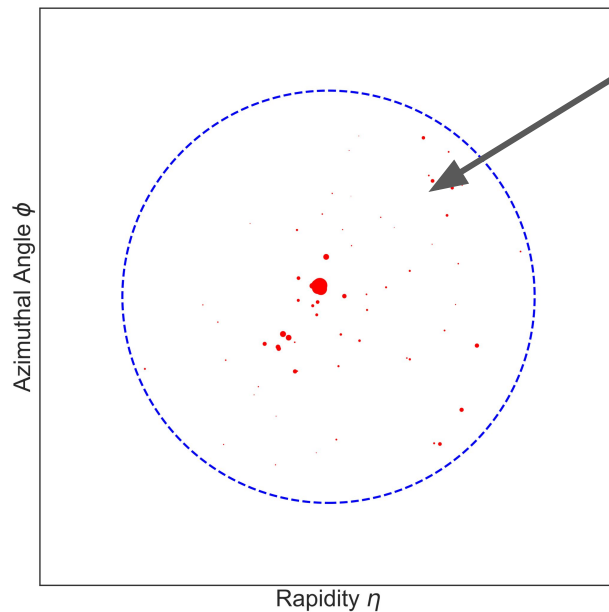


I don't mean the fixed size of a jet algorithm, e.g. AK4, but rather a dynamical notion of a jet size!



How “*big*” is this jet?

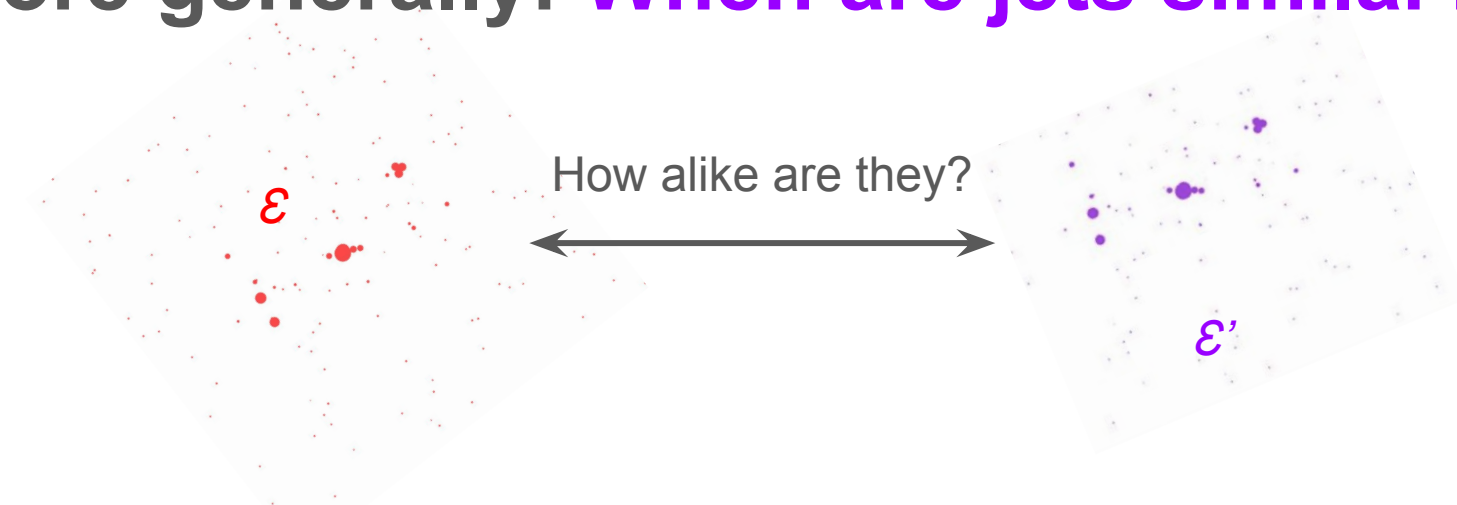
... What do we even mean by the “size” of a jet?



By following this line of thinking, we’ll be able to come up with a new class of QCD observables!

I don’t mean the fixed size of a jet algorithm, e.g. AK4, but rather a dynamical notion of a jet size!

More generally: When are jets similar?

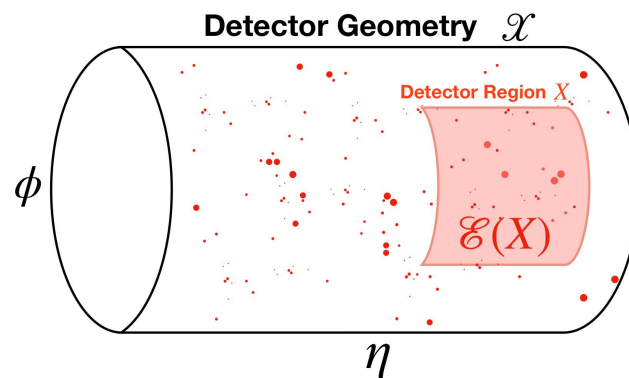


Jets can be represented as **point clouds** – let's scour through the ML and computer vision literature for a metric on point clouds we like!*

$$\mathcal{E}(y) = \sum_i z_i \delta(y - y_i)$$

Detector Coordinate (η, ϕ)

Energy Fraction E_i / E_{Tot}



*Or better yet, construct one!

More generally: When are jets similar?

How alike are they?

The **moment of clarity**:

Let's figure out what properties we want our similarity metric to satisfy, and then simply construct one satisfying all the properties!

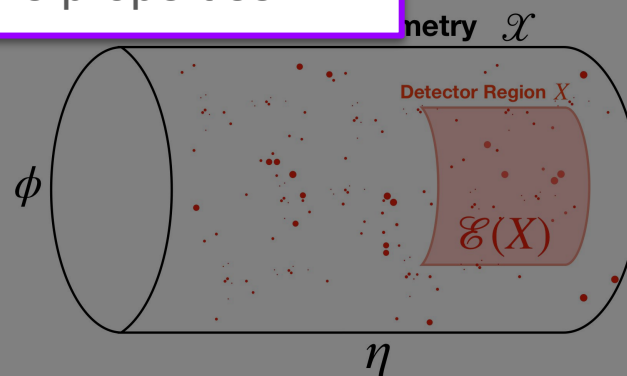
Jets can be compared

the ML and like!*

$$\mathcal{E}(y) = \sum_i z_i \delta(y - y_i)$$

Detector Coordinate (η, ϕ)

Energy Fraction E_i / E_{Tot}



*Or better yet, construct one!

The Wasserstein Metric

Let's demand the following reasonable properties of our metric on point clouds:

1. ... is a genuine metric: nonnegative, nondegenerate, finite
2. ... is **IRC-safe**^{*} (Calculable and robust)
3. ... is translationally invariant
4. ... is invariant to particle labeling
5. respects the detector metric **faithfully**^{**}

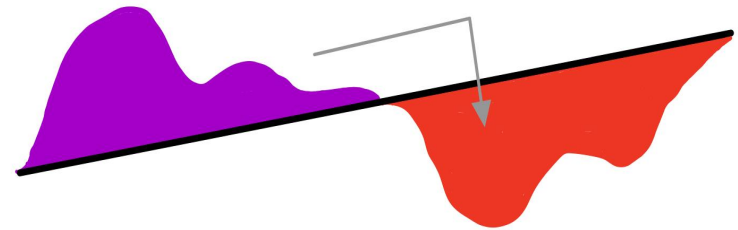
^{*}Ask me for more details on this offline!

^{**}Preserves distances between *extended* objects, not just points

The Wasserstein Metric

Let's demand the following reasonable properties of our metric on point clouds:

1. ... is a genuine metric: nonnegative, nondegenerate, finite
2. ... is **IRC-safe**^{*} (Calculable and robust)
3. ... is translationally invariant
4. ... is invariant to particle labeling
5. respects the detector metric **faithfully**^{**}



EMD = Work done to move “dirt” optimally

I can prove that the *only* metric that satisfies this is the **Wasserstein Metric / EMD**!

$$\text{EMD}^{(\beta, R)}(\mathcal{E}, \mathcal{E}') = \min_{\pi \in \mathcal{M}(\mathcal{X} \times \mathcal{X})} \left[\frac{1}{\beta R^\beta} \left\langle \pi, d(x, y)^\beta \right\rangle \right] + |\Delta E_{\text{tot}}|$$
$$\pi(\mathcal{X}, Y) \leq \mathcal{E}'(Y), \quad \pi(X, \mathcal{X}) \leq \mathcal{E}(X), \quad \pi(\mathcal{X}, \mathcal{X}) = \min(E_{\text{tot}}, E'_{\text{tot}})$$

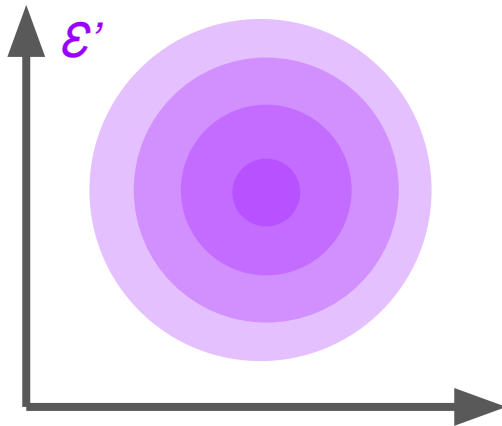
A staple of computer vision ML is also useful for jets!

Other common ML metrics, like Chamfer, MMDs, or Kernel metrics arent *faithful*.

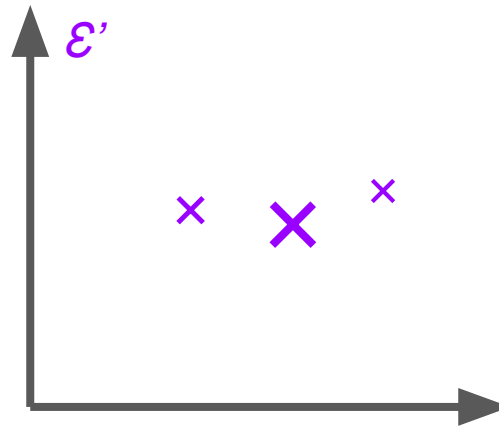
Shapes as Energy Flows

Energy flows don't have to be real events – they can be *any* energy distribution in detector space, or **shape**.

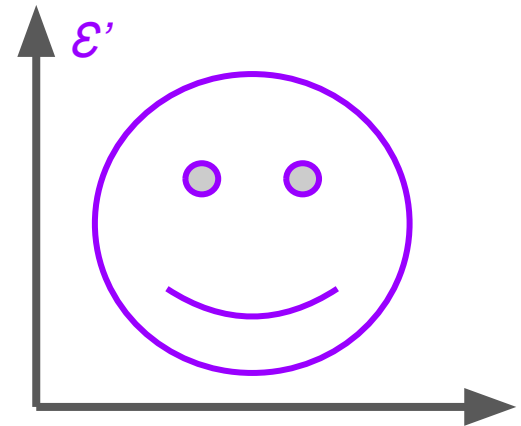
Can make anything you want! Even continuous or complicated shapes.
(Or, something you calculate in perturbative QCD)



Shape = 2D Gaussian



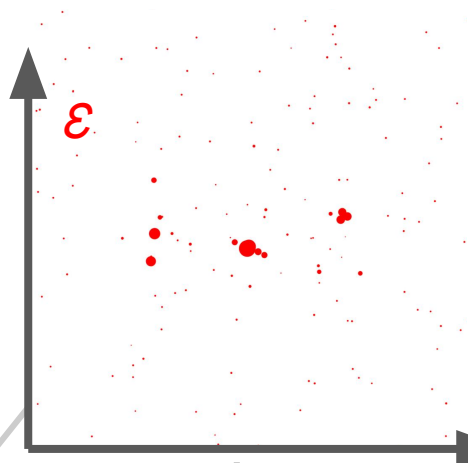
Shape = 3 Points



Shape = Smile

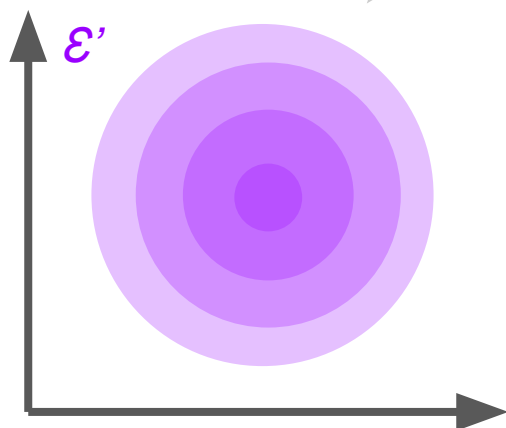
Shapiness

The EMD between a real event or jet $\mathcal{E}$ and idealized shape $\mathcal{E}'$ is the [shape]iness of $\mathcal{E}$ – a well defined IRC-safe observable!



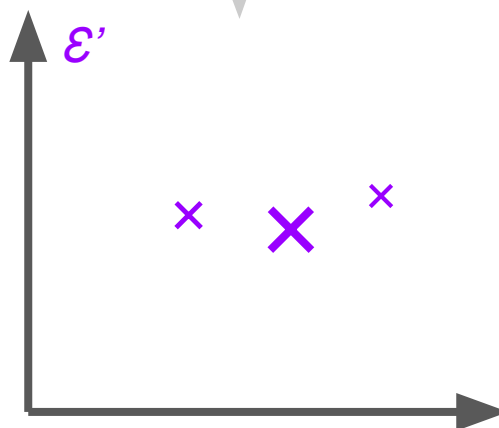
Answers the question:
“How much like the
shape $\mathcal{E}'$ is my event $\mathcal{E}$?”

Med EMD($\mathcal{E}, \mathcal{E}'$)
“Gaussiness”



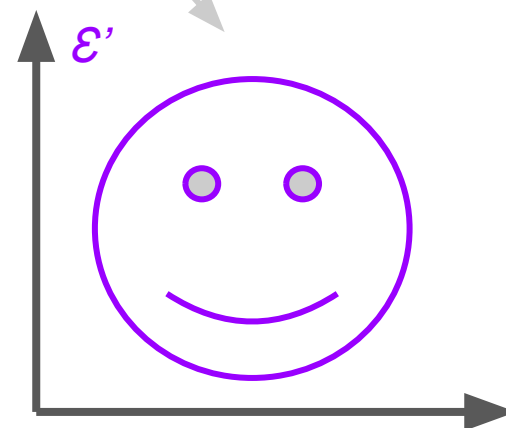
Shape = 2D Gaussian

Low EMD($\mathcal{E}, \mathcal{E}'$)
“3-Pointiness”
AKA “3-Subjettiness”



Shape = 3 Points

High EMD($\mathcal{E}, \mathcal{E}'$)
“Smileyness”



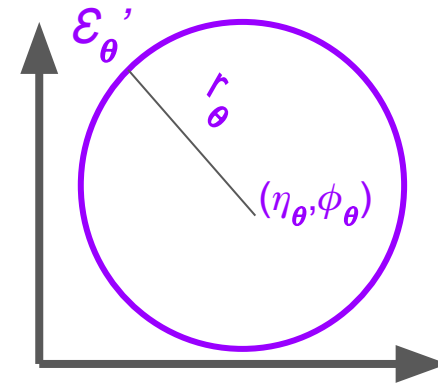
Shape = Smile

Mathematical Details - Shapiness

Rather than a single shape, consider a **parameterized manifold** $\mathcal{M}$ of **energy flows**.

e.g. The manifold of uniform circle energy flows:

$$\mathcal{E}_\theta'(y) = \begin{cases} \frac{1}{2\pi r_\theta} & |\vec{y} - \vec{y}_\theta| = r_\theta \\ 0 & |\vec{y} - \vec{y}_\theta| \neq r_\theta \end{cases}$$



Then, for an event $\mathcal{E}$, define the **shapiness** $\mathcal{O}_{\mathcal{M}}$ and **shape parameters** $\theta_{\mathcal{M}}$, given by:

$$\mathcal{O}_{\mathcal{M}}(\mathcal{E}) \equiv \min_{\mathcal{E}_\theta \in \mathcal{M}} \text{EMD}^{(\beta, R)}(\mathcal{E}, \mathcal{E}_\theta)$$

$$\theta_{\mathcal{M}}(\mathcal{E}) \equiv \operatorname{argmin}_{\mathcal{E}_\theta \in \mathcal{M}} \text{EMD}^{(\beta, R)}(\mathcal{E}, \mathcal{E}_\theta)$$

Observables $\Leftrightarrow$ Manifolds of Shapes

Observables can be specified solely by defining a **manifold of shapes**:

$$\mathcal{O}_{\mathcal{M}}(\mathcal{E}) \equiv \min_{\mathcal{E}_{\theta} \in \mathcal{M}} \text{EMD}^{(\beta, R)}(\mathcal{E}, \mathcal{E}_{\theta}),$$

$$\theta_{\mathcal{M}}(\mathcal{E}) \equiv \operatorname{argmin}_{\mathcal{E}_{\theta} \in \mathcal{M}} \text{EMD}^{(\beta, R)}(\mathcal{E}, \mathcal{E}_{\theta}),$$

Many well-known observables* already have this form!

Observable	Manifold of Shapes
N -Subjettiness	Manifold of N -point events
N -Jettiness	Manifold of N -point events with floating total energy
Thrust	Manifold of back-to-back point events
Event / Jet Isotropy	Manifold of the single uniform event

... and more!

All of the form “How much like **[shape]** does my **event** look like?”

Generalize to *any* shape.

*These observables are usually called event shapes or jet shapes in the literature – we are making this literal!

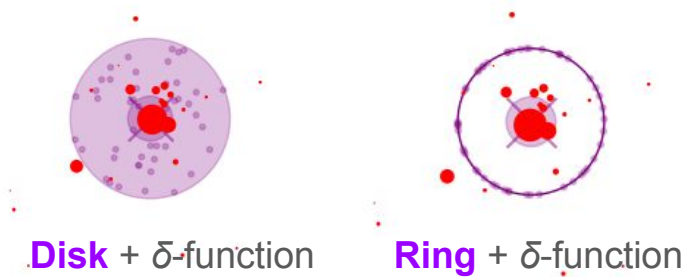
Hearing Jets Ring

(and Disk, and Ellipse)

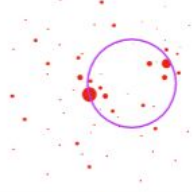
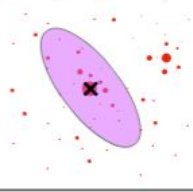
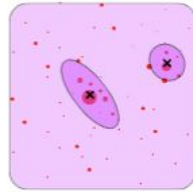
$\mathcal{O}_{\mathcal{M}}(\mathcal{E})$ answers: “How much like a **shape** in $\mathcal{M}$ does my **event** $\mathcal{E}$ look like?”

$\theta_{\mathcal{M}}(\mathcal{E})$ answers: “Which **shape** in $\mathcal{M}$ does my **event** $\mathcal{E}$ look like?”

Can define complex manifolds to probe increasingly subtle geometric structure!



Probing **collinear** (δ -function) and **soft** (shape) structure!

Shape	Specification	Illustration
Ringiness $\mathcal{O}_R$	Manifold of Rings $\mathcal{E}_{x_0,R_0}(x) = \frac{1}{2\pi R_0}$ for $ x - x_0 = R_0$ x_0 = Center, R_0 = Radius	
Diskiness $\mathcal{O}_D$	Manifold of Disks $\mathcal{E}_{x_0,R_0}(x) = \frac{1}{\pi R_0^2}$ for $ x - x_0 \leq R_0$ x_0 = Center, R_0 = Radius	
Ellipsiness $\mathcal{O}_E$	Manifold of Ellipses $\mathcal{E}_{x_0,a,b,\varphi}(x) = \frac{1}{\pi ab}$ for $x \in \text{Ellipse}_{x_0,a,b,\varphi}$ x_0 = Center, a, b = Semi-axes, φ = Tilt	
(Ellipse Plus Point)iness	Composite Shape $\mathcal{O}_E \oplus \tau_1$ Fixed to same center x_0	
N-(Ellipse Plus Point)iness Plus Pileup	Composite Shape $N \times (\mathcal{O}_E \oplus \tau_1) \oplus \mathcal{I}$	

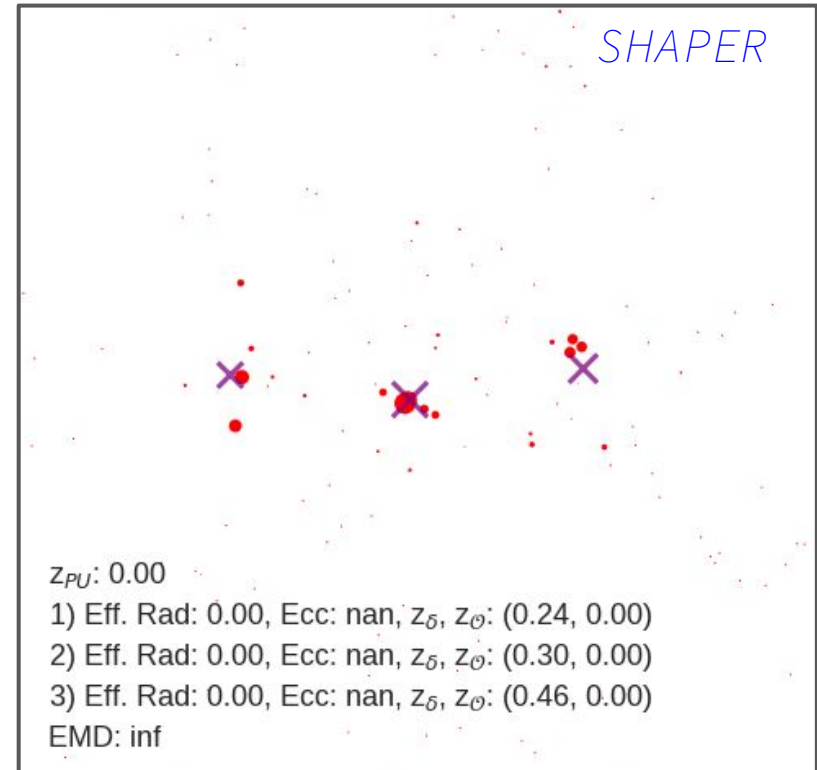
Some examples of new shapes you can define!

New IRC-Safe Observables

The *SHAPER* framework makes it easy to algorithmically invent new jet observables!

e.g. *N-(Ellipse+Point)iness+Pileup* as a jet algorithm:

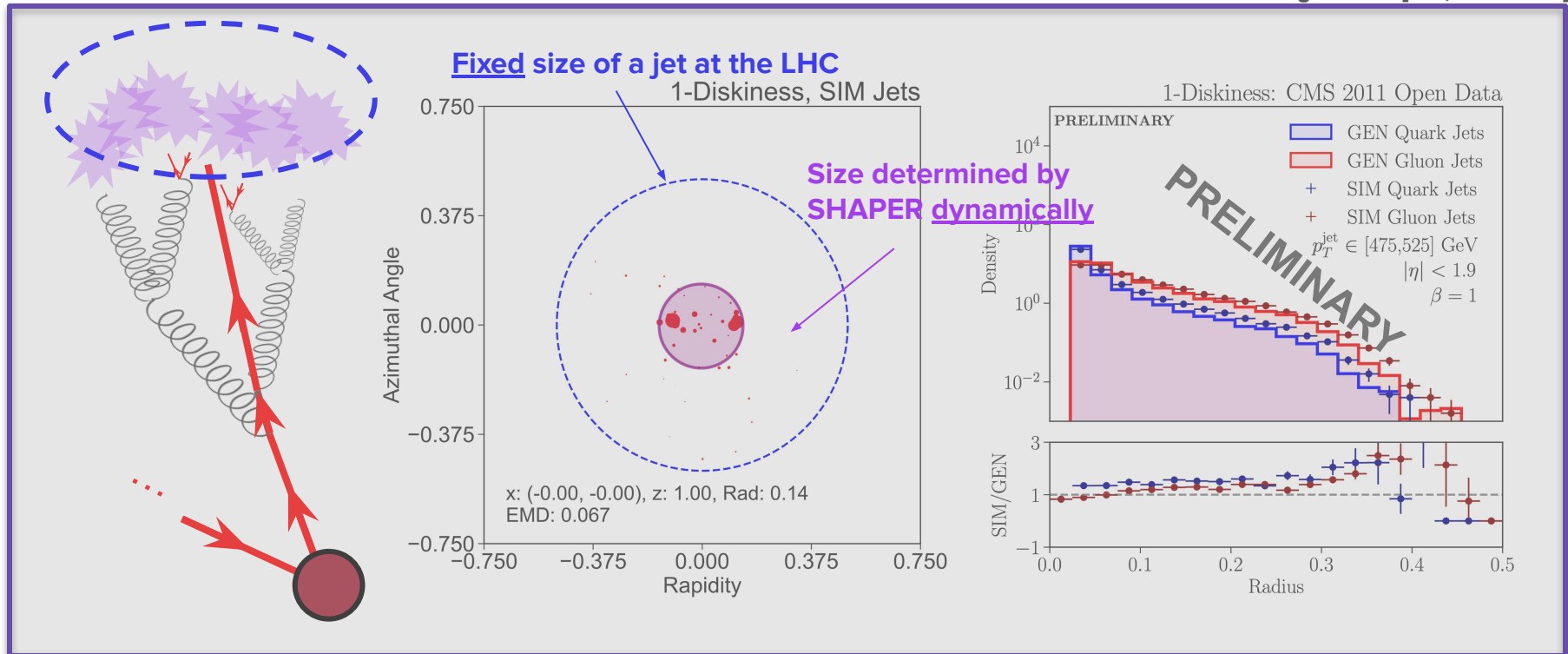
- Learn jet centers + collinear radiation
- Dynamic jet radii (no R parameter!)
- Dynamic eccentricities and angles
- Dynamic jet energies
- Dynamic pileup (no z_{cut} parameter!!!)
- Learned parameters for discrimination



Can design custom specialized jet algorithms to learn jet substructure!

Back to our question: How Big are Jets?

Slide images from [Wu; APS 2023]

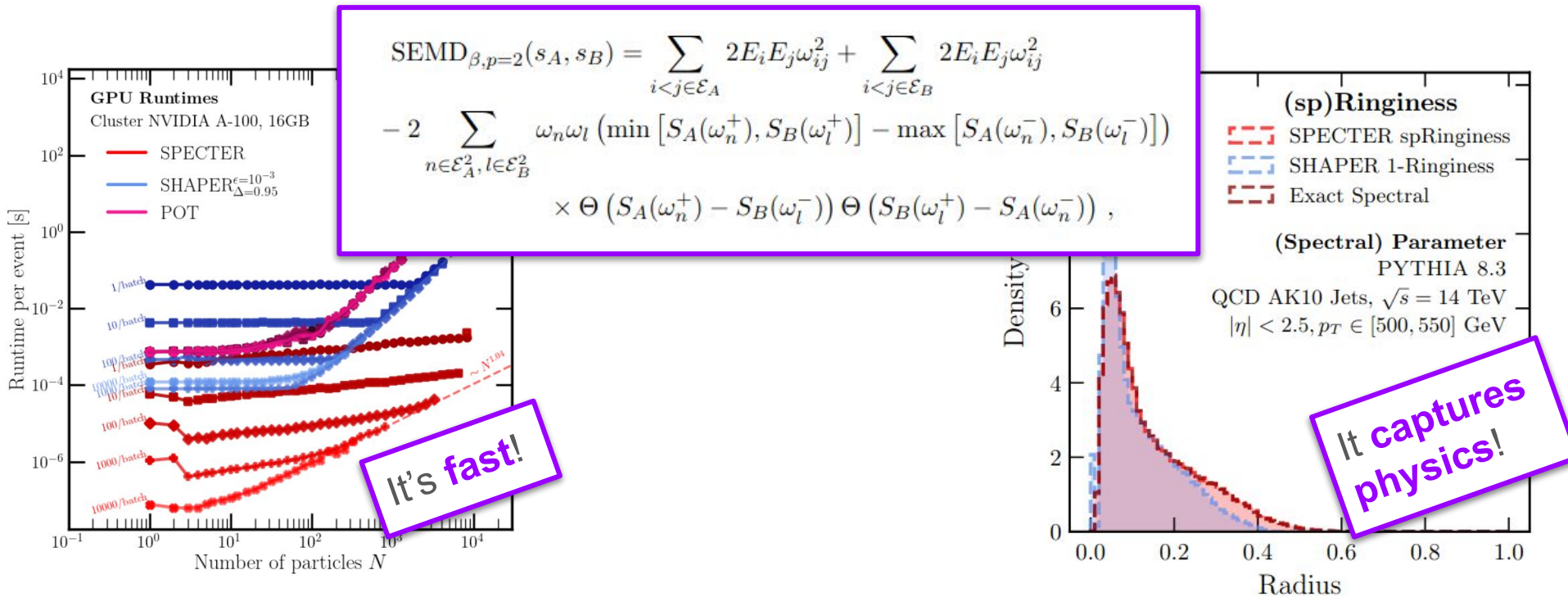


We can now answer this question in a precise way!



SPECTER: We can do more with OT!

By demanding our loss is only *almost** always a metric on events rather than always a metric, we can build a new loss that is *completely solvable*!



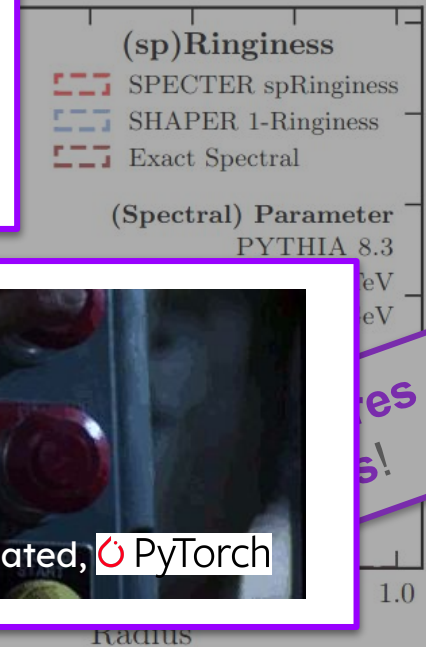
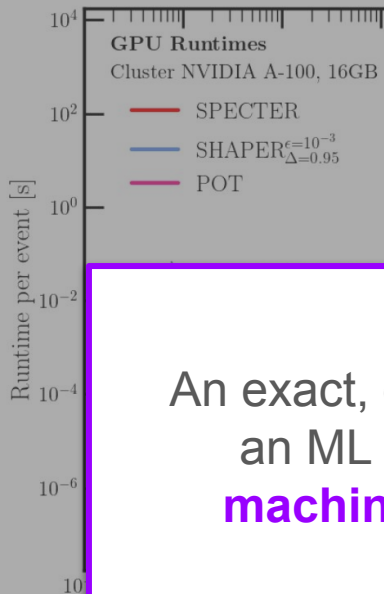
*By *almost*, I mean that there is a measure-0 set of events for which $d(x,y) = 0$ does not imply $x = y$
Ask me later about connections to EECs!



SPECTER: We can do more with OT!

By demanding our loss is only *almost*^{*} always a metric on events rather than always a metric, we can build a new loss that is *completely solvable*!

$$\begin{aligned} \text{SEMD}_{\beta,p=2}(s_A, s_B) = & \sum_{i < j \in \mathcal{E}_A} 2E_i E_j \omega_{ij}^2 + \sum_{i < j \in \mathcal{E}_B} 2E_i E_j \omega_{ij}^2 \\ & - 2 \sum_{n \in \mathcal{E}_A^2, l \in \mathcal{E}_B^2} \omega_n \omega_l \left(\min [S_A(\omega_n^+), S_B(\omega_l^+)] - \max [S_A(\omega_n^-), S_B(\omega_l^-)] \right) \\ & \times \Theta (S_A(\omega_n^+) - S_B(\omega_l^-)) \Theta (S_B(\omega_l^+) - S_A(\omega_n^-)) , \end{aligned}$$



An exact, closed form for what started as an ML problem ... can we take the **machine** out of **machine learning**?



[Cameron et. al.; 1984]

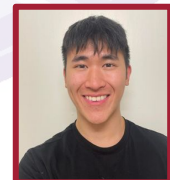
^{*}By *almost*, I mean that there is a measure-0 set of events for which $d(x,y) = 0$ does not imply $x = y$
Ask me later about connections to EECs!

Part III

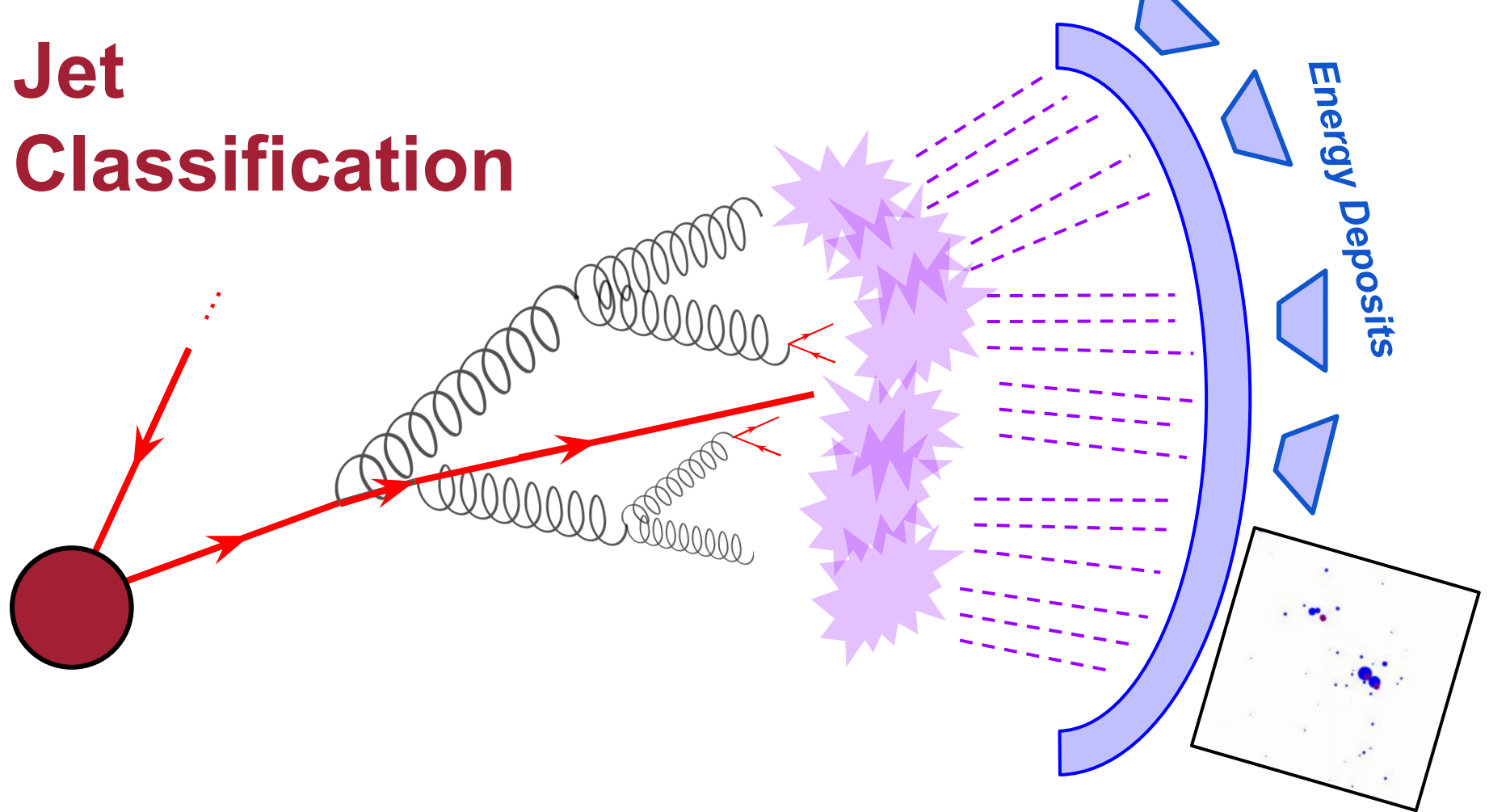
Moments of Clarity: Streamlining Latent Spaces



Download
Moment EFNs!



Jet Classification

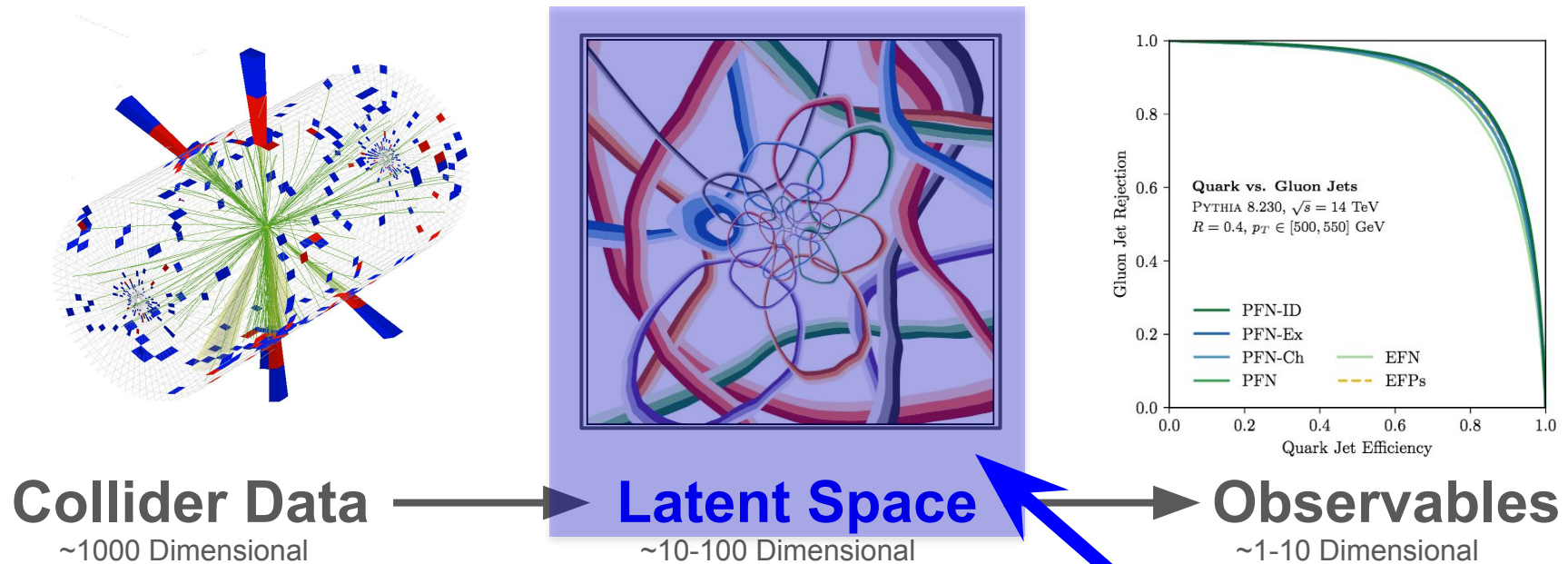


Given the final measured **point cloud**, was the **initiating parton** a **quark** or a **gluon**?

A staple machine learning task!

Rikab Gambhir – Brown – 17 October 2024

Typical Machine Learning Setup



Pictured: An Energy Flow Network (**EFN**):

$$\mathcal{O}(\{p_1, \dots, p_M\}) = F \left(\sum_{i=1}^M z_i \Phi(\hat{p}_i) \right)$$

(How) can we understand and constrain this?

(How) can we be more efficient?

Deep Learning on Point Clouds

The **Deep Sets Theorem** tells us how to parameterize functions on point clouds*:

Set of momenta $\rightarrow$ $a = 1 \dots L$, the *Latent Dimension* index

$$\mathcal{O}(\mathcal{P}) = F(\langle \phi^a \rangle_{\mathcal{P}})$$

The Deep Sets Theorem guarantees that any function on sets $\mathcal{P}$ can be written this way, for “sufficiently complex” F and ϕ and large enough L .

$\langle \phi \rangle_{\mathcal{P}} \equiv \sum_i z_i \phi(\hat{p}_i)$

For this talk, I will focus primarily on Energy Flow Networks (EFNs) where the sums are energy-weighted.

*More sophisticated architectures still reduce to Deep Sets in some limit, e.g. graph networks

Deep Learning on Point Clouds

The **Deep Sets Theorem** tells us how to parameterize functions on point clouds*:

Set of momenta $\rightarrow$ $a = 1 \dots L$, the *Latent Dimension* index

$$\mathcal{O}(\mathcal{P}) = F(\langle \phi^a \rangle_{\mathcal{P}})$$

The Deep Sets Theorem guarantees that any function on sets $\mathcal{P}$ can be written this way, for “sufficiently complex” F and ϕ and large enough L .

$\langle \phi \rangle_{\mathcal{P}} \equiv \sum_i z_i \phi(\hat{p}_i)$

But how complex do F and ϕ need to be? Can I reduce the number of latent dimensions?

For this talk, I will focus primarily on Energy Flow Networks (EFNs) where the sums are energy-weighted.

*More sophisticated architectures still reduce to Deep Sets in some limit, e.g. graph networks

Moment Pooling

$$\mathcal{O}(\mathcal{P}) = F(\langle \phi^a \rangle_{\mathcal{P}})$$



Generalize to more moments! “**Moment Pooling**”

$$\mathcal{O}_k(\mathcal{P}) = F_k(\underbrace{\langle \phi^a \rangle_{\mathcal{P}}}_{1^{\text{st}} \text{ Moment}}, \underbrace{\langle \phi^{a_1} \phi^{a_2} \rangle_{\mathcal{P}}}_{2^{\text{nd}} \text{ Moment}}, \dots, \underbrace{\langle \phi^{a_1} \dots \phi^{a_k} \rangle_{\mathcal{P}}}_{k^{\text{th}} \text{ Moment}})$$

k = Highest order moment considered

$$L_{\text{eff}} = \binom{L+k}{k}$$

Moment Pooling – Why?

k = Highest order moment considered ↓

$$\mathcal{O}_k(\mathcal{P}) = F_k \left(\underbrace{\langle \phi^a \rangle_{\mathcal{P}}}_{1^{\text{st}} \text{ Moment}}, \underbrace{\langle \phi^{a_1} \phi^{a_2} \rangle_{\mathcal{P}}}_{2^{\text{nd}} \text{ Moment}}, \dots, \underbrace{\langle \phi^{a_1} \dots \phi^{a_k} \rangle_{\mathcal{P}}}_{k^{\text{th}} \text{ Moment}} \right)$$

$L_{\text{eff}} = \binom{L+k}{k}$

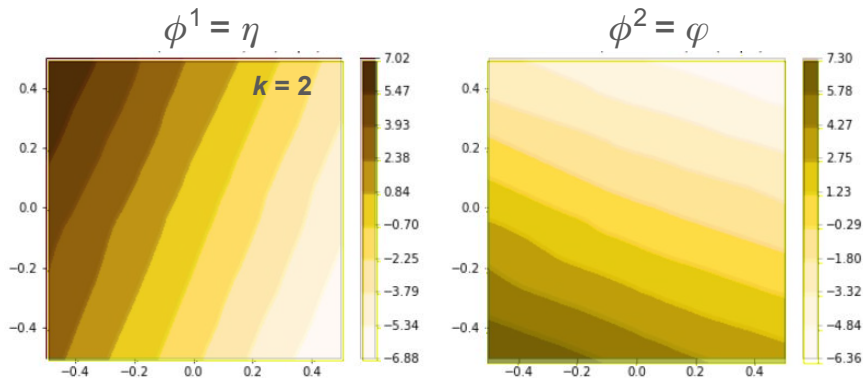
- **Explicit Multiplication:** Neural nets are mostly piecewise linear! But most functions we learn aren't. Moments are just multiplication, but for distributions!
- **Latent Space Compression:** For large L , there are effectively L^k latent dimensions due to combinatorics, but still made of only L functions! Fewer latent dimensions means easier analysis!

Multiplication is all You Need

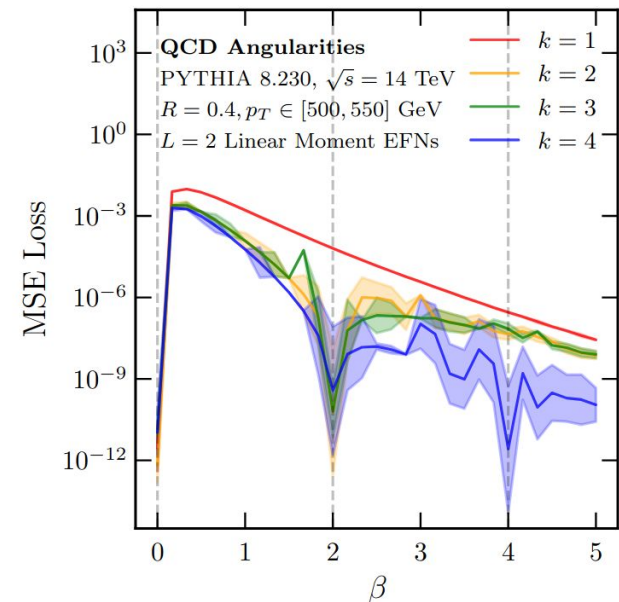
In the moment language, even integer^{*} β jet angularities $\Leftrightarrow k = \beta^{\text{th}}$ moments!

$$\begin{aligned}\lambda^{(\beta)}(\mathcal{P}) &= \sum_i z_i (\eta_i^2 + \phi_i^2)^{\beta/2} \\ &= \langle \eta^\beta \rangle_{\mathcal{P}} + \langle \phi^\beta \rangle_{\mathcal{P}} + \text{Cross Moments}\end{aligned}$$

Learns the simplest latent representations!

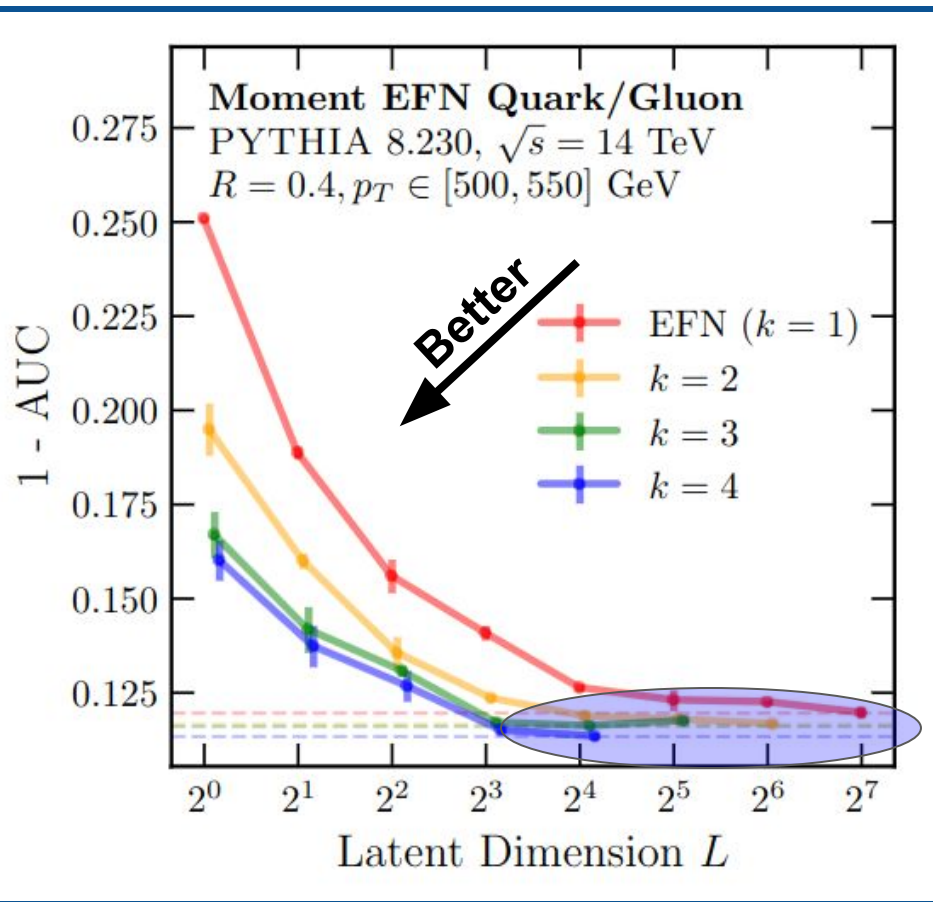


Encourage learning a diagonal basis in (η, ϕ) using L1 Regularization on F



The lesson: Basic observables are naturally represented with multiplication, so build it in!

Moment Pooling Results

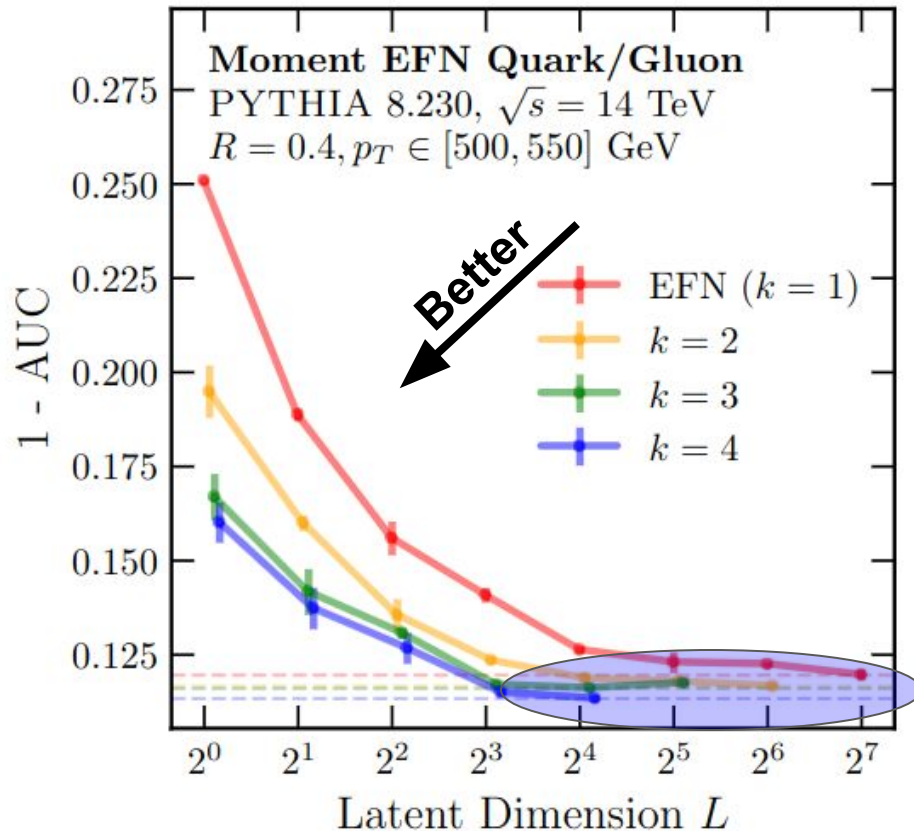


A high order **Moment EFN** can achieve the same top performance on quark/gluon classification as an **ordinary EFN**, but with far fewer latent dimensions!*

A Moment EFN is a more efficient encoding of the same data!

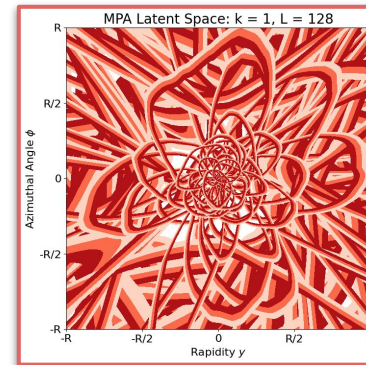
*Interestingly, the performance is constant in the *effective* latent dimensions and number of parameters. Ask me about this!

The Black Box

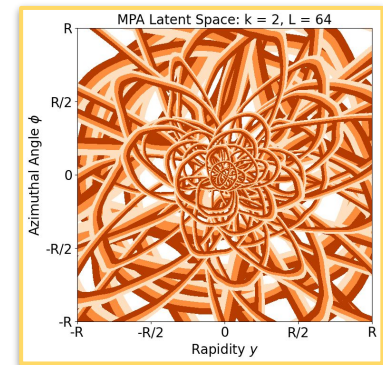


Latent Spaces

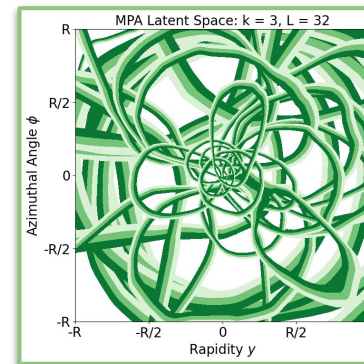
EFN ($k = 1$)
 $L = 128$



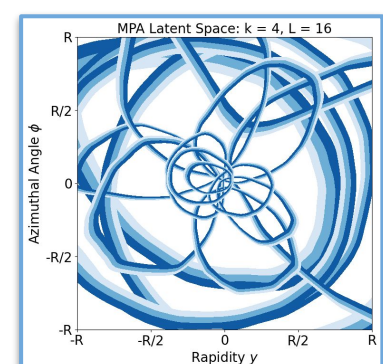
$k = 2$
 $L = 64$



$k = 3$
 $L = 32$



$k = 4$
 $L = 16$

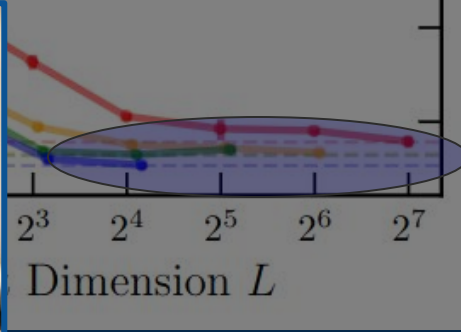


The 45% - 55% contours of each of the L different ϕ functions

*Interestingly, the performance is constant in the *effective* latent dimensions and number of parameters. Ask me about this!

So what? The information is just being shuffled around. Is 16 latent dimensions any more interpretable than 128?

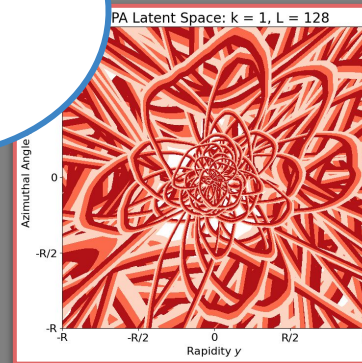
The machine probably learned something, but I didn't!



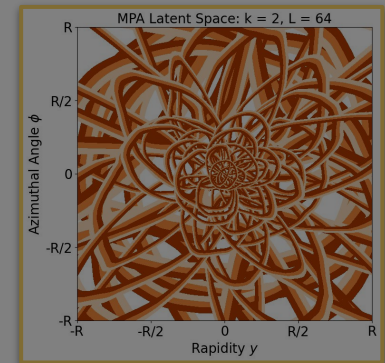
Me, just before the **moment** of clarity.

Latent Spaces

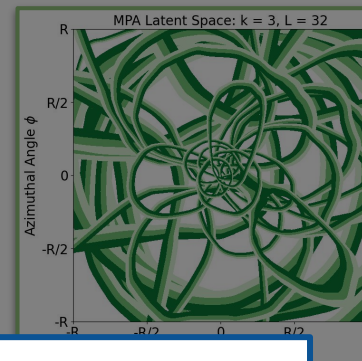
EFN ($k = 1$)
 $L = 128$



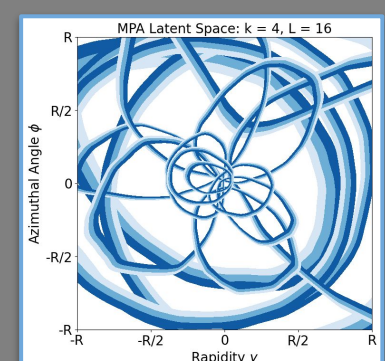
$k = 2$
 $L = 64$



$k = 3$
 $L = 32$



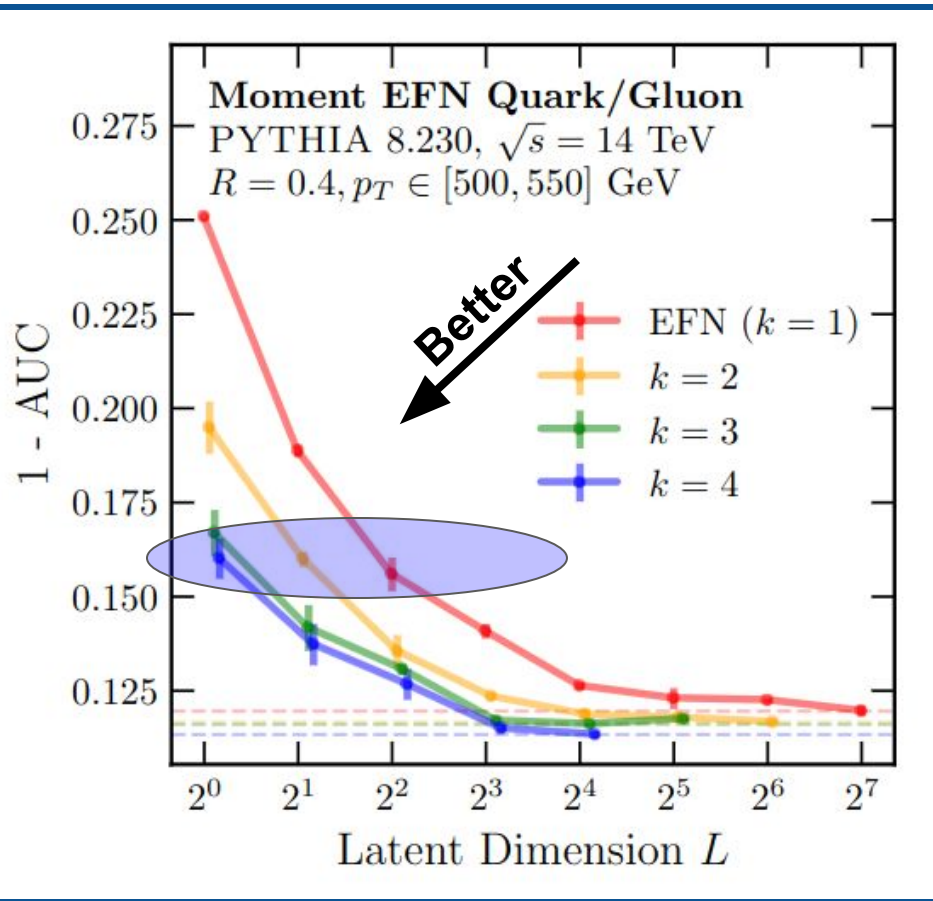
$k = 4$
 $L = 16$



trajectories of each of the L different ϕ functions

*Interestingly, the performance is constant in the *effective* latent dimensions and number of parameters. Ask me about this!

Moment Pooling Results (Cont'd)



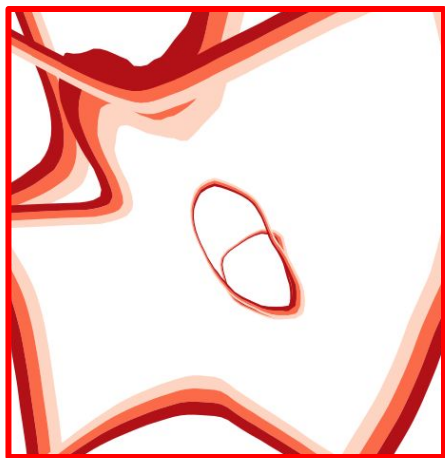
Let's instead look at networks with just a *single* latent dimension.

A **$k = 4$ Moment EFN** with *just one* latent dimension does just as good as an **ordinary EFN** with *four* latent dimensions!

Going down to $L = 1$

The more efficient encoding is extremely useful when we can get to $L = 1$!
Look how much simpler allowing neural networks to multiply makes things!

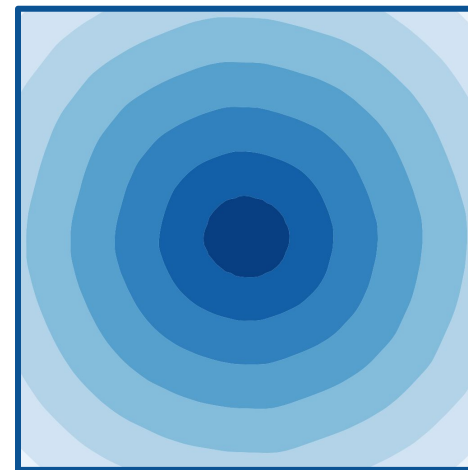
EFN ($k = 1$)
 $L = 4$



Same information!



Order $k = 4$
 $L = 1$



- 4 Different Functions
- Combined in a complicated way
- No symmetry
- ???

- Single function
- Expected to be, and is, radially symmetric
- We can **interpret** this!

The Moment of Truth

We can extract physics from this!

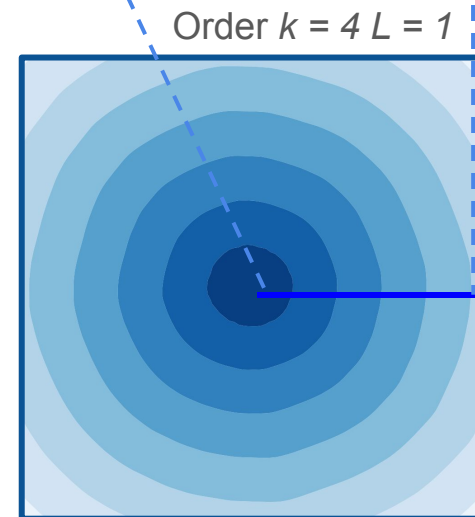
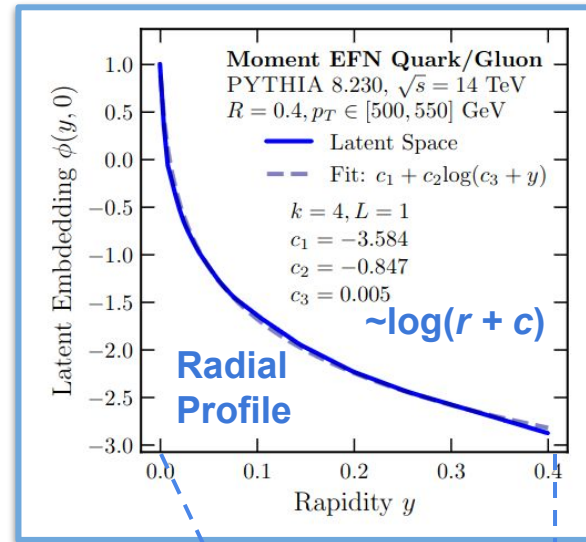
$$\Phi_{\mathcal{L}}(r) = c_1 + c_2 \log(c_3 + r)$$

This defines a **log angularity** observable, related to the $\square \rightarrow 0$ limit of ordinary angularities

We can even see non-perturbative physics!

$$c_3 \sim \frac{\Lambda_{\text{QCD}}}{p_T R} \sim 0.001$$

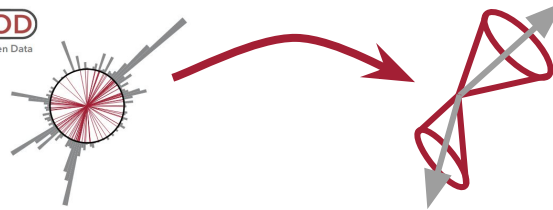
Just this alone is enough to get an IRC-safe quark-gluon classifier with an AUC ~ 0.83 !



Part IV: Conclusion

Learning **Uncertainties** the
Frequentist Way

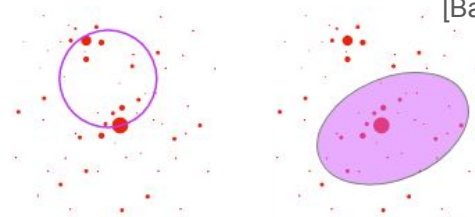
MOD
MIT Open Data



[RG, Nachman, Thaler,
[2205.05084](#)

[RG, Nachman, Thaler,
[2205.05084](#)]

Can you Hear the **Shape** of a
Jet?

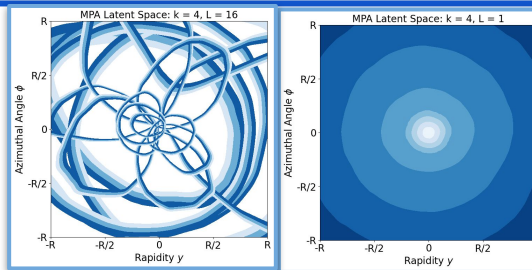


[Ba, Dogra, RG, Tasissa, Thaler,
[2302.12266](#)]

[RG, Thaler, Wu, WIP]

[RG, Larkoski, Thaler,
[2410.05379](#)]

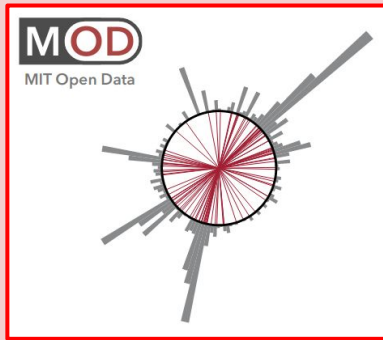
The **Moments** of Clarity:
Streamlining Latent Spaces



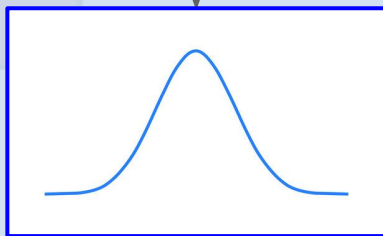
[RG, Osathapan,
Thaler, [2403.08854](#)]

Things a **machine** can understand

Sophisticated Detector Models

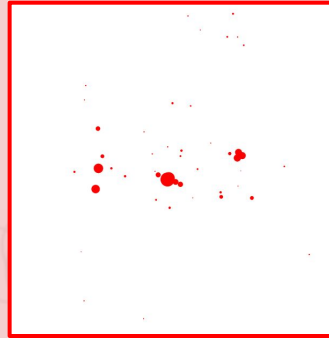


Using the
Gaussian
Ansatz and
DV Loss

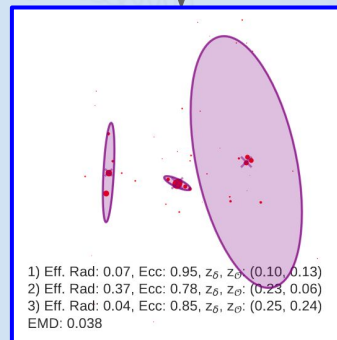


Gaussians

Complicated point clouds

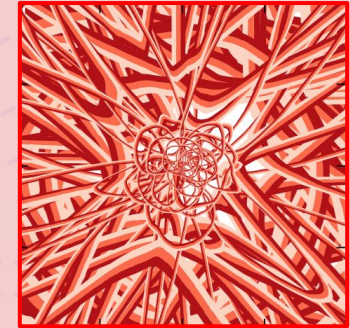


Using faithful
optimal transport

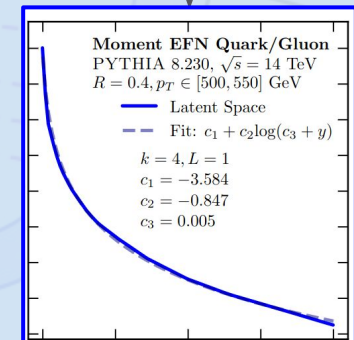


Basic Kindergarten Shapes

Huge Latent Spaces



Using Moment
Pooling

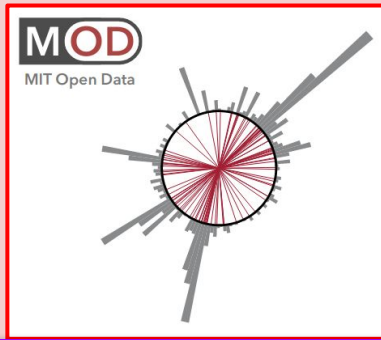


Addition, multiplication, and a
few elementary functions

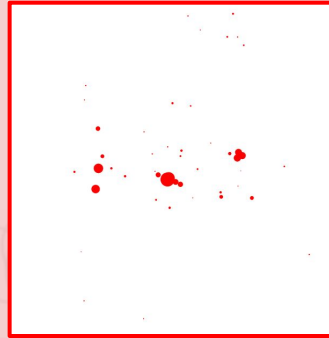
Things my **mortal physicist brain** can understand

Things a **machine** can understand

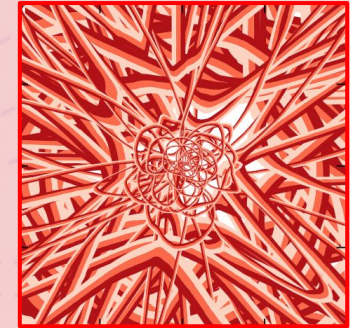
Sophisticated Detector Models



Complicated point clouds

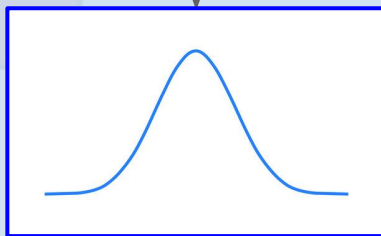


Huge Latent Spaces



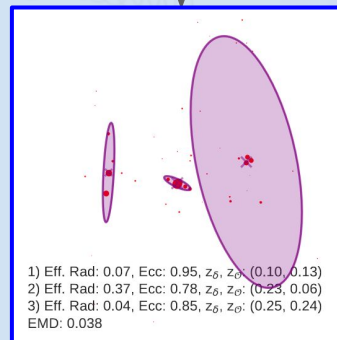
Main Takeaway: By using purpose-built machine learning architectures and losses, we can make sure we extract the **physics we want** from our machines!

Using the
Gaussian
Ansatz and
DV Loss



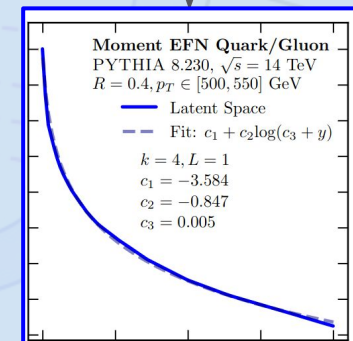
Gaussians

Using faithful
optimal transport



Basic Kindergarten Shapes

Using Moment
Pooling



Addition, multiplication, and a few elementary functions

Things my **mortal physicist brain** can understand

Email me questions at rikab@mit.edu!

[RG, Nachman, Thaler; [2205.03413](#)]

[RG, Nachman, Thaler; [2205.05084](#)]

[Ba, Dogra, RG, Tasissa, Thaler; [2302.12266](#)]

[RG, Thaler, Wu; WIP]

[RG, Larkoski, Thaler; [2410.05379](#)]

[RG, Osathapan, Thaler; [2403.08854](#)]

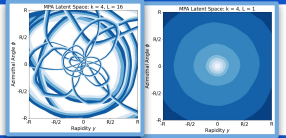
Learning **Uncertainties** the
Frequentist Way



Can you Hear the **Shape** of a
Jet?

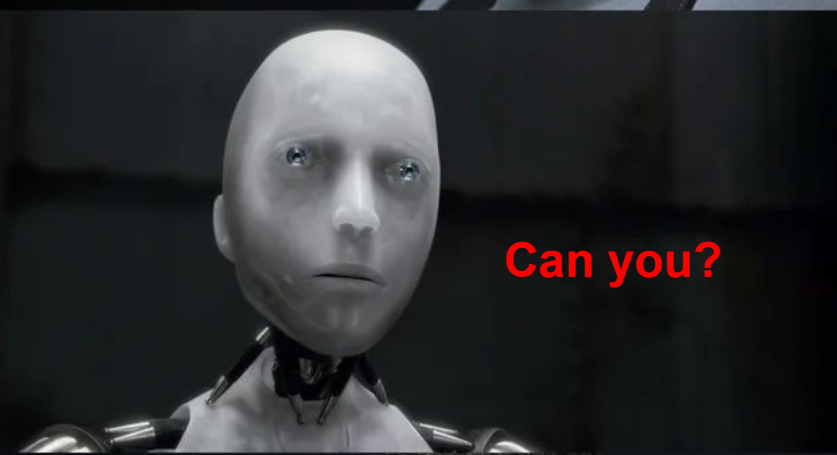


The **Moments** of Clarity:
Streamlining Latent Spaces



Any Questions?

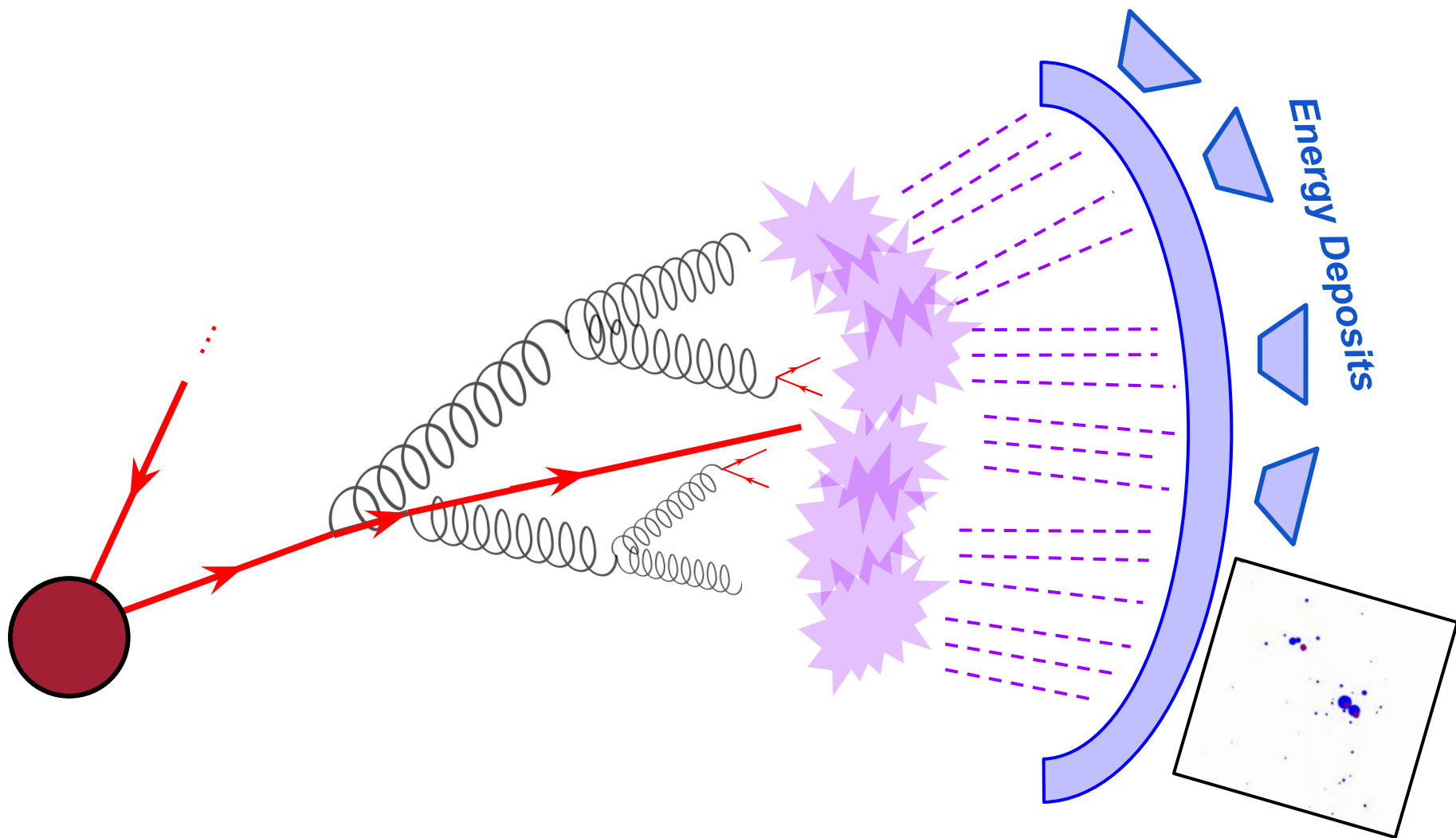
Can a robot understand jet physics?
Can it understand QCD?



Can you?



Backup



Learning MLC

How do we calculate f ?

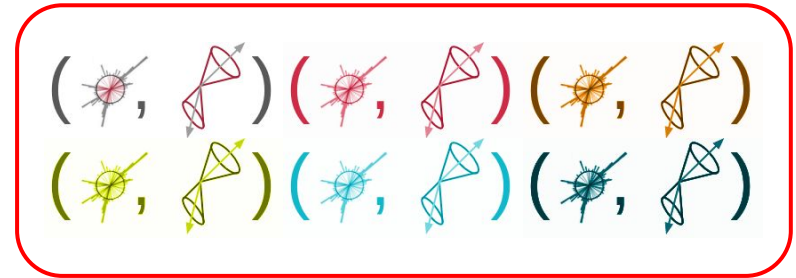
$$\begin{aligned} f_{\text{MLC}}(x) &= \operatorname{argmax}_z p_{\text{train}}(x|z) \\ &= \operatorname{argmax}_z \log \underbrace{\frac{p_{\text{train}}(x, z)}{p_{\text{train}}(x)p_{\text{train}}(z)}}_{T(x, z)} \end{aligned}$$

The function T is the likelihood ratio between $p(x, z)$ and $p(x)p(z)$.

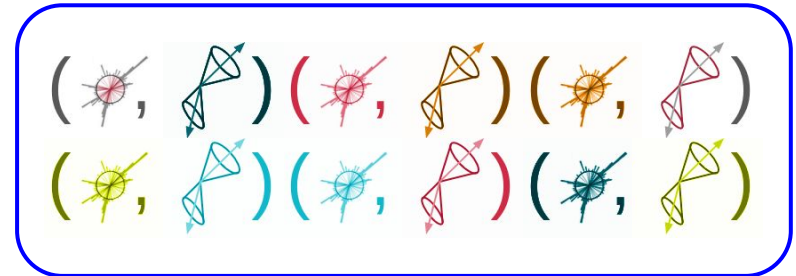
↓
Neyman–Pearson

T is the **optimal classifier** between (x, z) pairs and shuffled (x, z) pairs!

Class P



Class Q



Classify between **P** and **Q**!

Aside: Mutual information

A measure for non-linear interdependence is the **Mutual Information**:

$$\begin{aligned} I(X; Z) &= \int dx dz p(x, z) \log \frac{p(x, z)}{p(x) p(z)} \\ &= \mathbb{E}_{\text{train}} T(X, Z) \end{aligned}$$

Answers the question: How much information, in terms of bits, do you learn about Z when you measure X (or vice versa)?

When doing calibration this way, we get a measure of the **correlation** between X and Z , *for free*.

Learning T

The **Donsker-Varadhan Representation (DVR)** of the KL divergence has been used in the statistics literature for mutual information estimation

$$\mathcal{L}_{\text{DVR}}[T] = - \left(\mathbb{E}_{P_{XZ}} [T] - \log \left(\mathbb{E}_{P_X \otimes P_Z} [e^T] \right) \right)$$

Interestingly, a nonlocal loss!

Strict bound on $I(X;Z)$

Minimized when

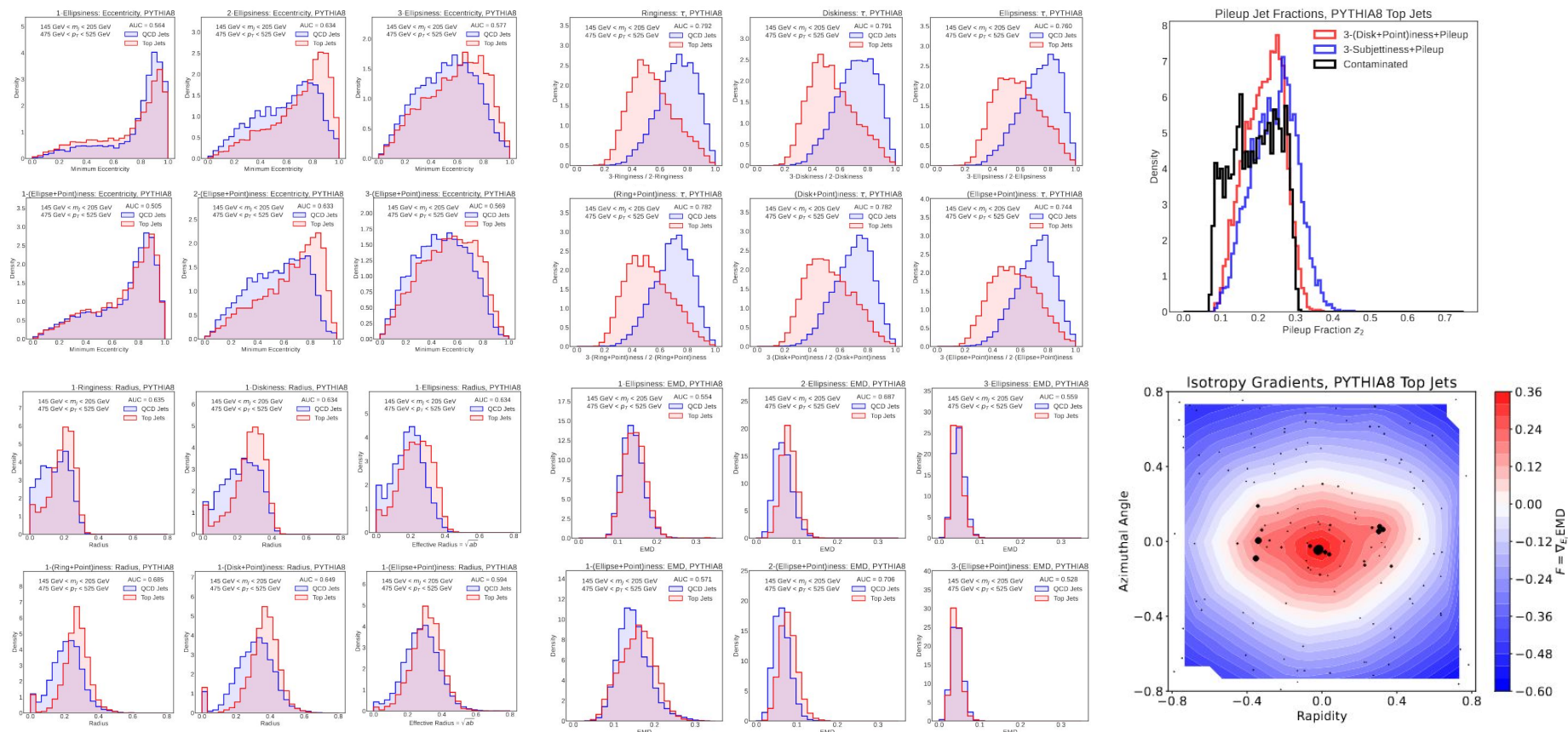
$$T(x, z) = \log \frac{p(x|z)}{p(x)} + c$$

What we want!

Unimportant

Lots of other losses also work, but DVR has very nice convergence properties - ask me later!

New IRC-Safe Observables



... **Lots** of extractable information!

Automatic Grooming with Shapes

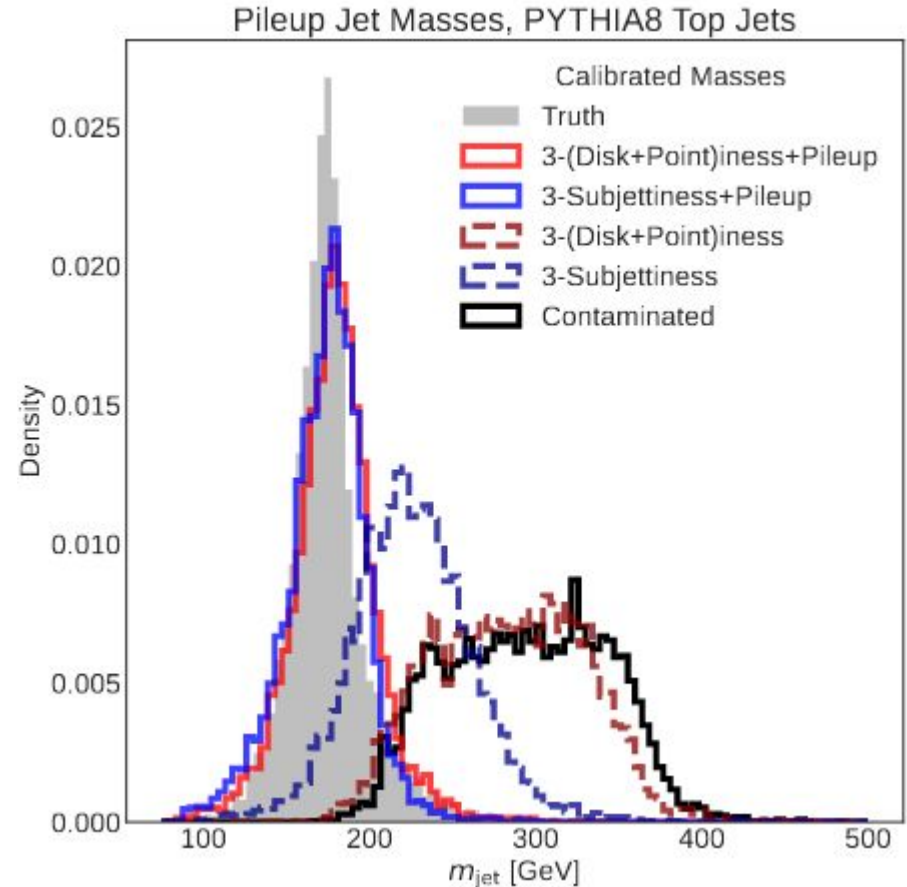
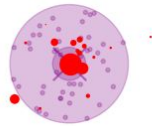
Use shapes to approximate events and extract masses – model pileup with a uniform background with floating weight!

No external hyperparameters, unlike softdrop. Only need to assume pileup is uniform!

Contaminate top jets with 5-30% extra energy spread uniformly in an 0.8×0.8 plane

Consider 4 shapes:

- 3-Subjettiness
- 3-Subjettiness + Pileup
- 3-(Disk+Point)iness ←
- 3-(Disk+Point)iness + Pileup



Can also consider ellipses instead of disks – only marginally better performance

